

BernatLab

IA local amb Ollama: RAG, veu i privadesa

Raspberry Pi · Docker · IA · LoRa · Hort Osona

Mòdul 4

Bernat — 8 de juliol del 2026

Document tècnic de consulta personal

Sobre aquest manual

Aquest és el segon mòdul del manual tècnic del BernatLab. Cobreix tota la cadena de sensors i visualització: el protocol MQTT, el broker Mosquitto, l'esquema de publicació dels sensors, la base de dades InfluxDB, l'agent Telegraf, la programació visual amb Node-RED, la visualització amb Grafana, l'API REST amb FastAPI, la integració amb la web pública Hort Osona i l'operativa del dia a dia.

Com es llegeix

En ordre, si parteixes de zero. Per capítols, si ja tens una base i vols aprofundir. Cada capítol segueix la mateixa estructura: teoria, aplicació al BernatLab, esquemes, comandes útils, errors habituals, exercicis pràctics i resum final.

Índex de capítols

- 11. Del Mòdul 1 al M2: què construïm
- 12. MQTT des de zero
- 13. Mosquitto al BernatLab
- 14. Publicar dades: els sensors
- 15. InfluxDB: base de dades de sèries temporals
- 16. Telegraf: el pont
- 17. Node-RED: programació visual
- 18. Fluxos pràctics
- 19. Grafana: visualitzar les dades
- 20. API pública: servir les dades al món
- 21. Integració amb Hort Osona
- 22. Operativa: còpies, alertes, escalat

Pròleg — Com s'ha fet aquest llibre

*“Si una eina és bona per ensenyar, també és bona per ensenyar-se a fer servir.”**

Aquest pròleg és una confessió. Explica com s'ha generat el llibre que tens a les mans, per què, i quines limitacions té. És tan pedagògic com la resta, perquè la millor manera d'aprendre a fer servir la IA local és veure com s'ha fet servir de veritat.

Les eines

Tres peces:

- **Ollama** al Mac (no pas a la Raspberry). És el motor que executa el model de llenguatge local. Si llegeixes el M4, ja saps què és.
- **Hermes** com a entorn de treball. Un assistent d'escriptura i programació que connecta amb el model i permet iterar, validar, i executar coses.
- **MiniMax-M3:Cloud** com a model de raonament. És el "cervell" que ha escrit la majoria dels capítols. La part de "Cloud" al nom vol dir que el model corre al núvol del proveïdor (no pas local al Mac), tot i que el plantejament del BernatLab defensa la IA local — i això és una de les limitacions que explico més avall.

El procés

El flux ha estat iteratiu, no pas lineal:

1. **Planificació:** l'autor (Bernat Mora) decidia el temari, l'estructura per mòduls, els capítols concrets. Això ho feia jo (l'assistent), proposant opcions i ell triant.
2. **Escriptura:** jo escrivia cada capítol seguint les 8 seccions estàndard (teoria, aplicació al BernatLab, esquemes, comandes, "què està passant realment", errors habituals, exercicis, resum). En una sola passada, sense revisions llargues.
3. **Validació amb execució real:** aquí ve la part important. Jo generava, però l'autor validava. Per exemple, jo proposava la instal·lació de Grafana; ell provava i em deia "el port 3000 no és correcte, és el 3000 sí" o "la comanda X falla perquè Debian 13 ja no té /var/log/auth.log". Jo corregia i re-generava.
4. **Iteració sobre artefactes:** PDF, DOCX, HTML. Cada vegada que canviava un capítol, jo regenerava tots els artefactes amb el meu propi `make_book.py` (sí, el codi que genera el llibre està al llibre — un bucle bonic).
5. **Publicació:** Git, GitHub Pages, commit, push. Tot automàtic.

El que la IA fa bé en aquest procés

- **Velocitat:** escriure 562 pàgines en poques setmanes seria impossible a mà.
- **Densitat:** jo no tinc la temptació d'omplir pàgines amb coses evidents. Si una secció és buida, s'elimina.

- **Replicabilitat:** el procés és 100% reproduïble. Si vols fer el teu propi llibre tècnic amb IA, ja tens la recepta.
- **Coherència:** el to, l'estructura, i el nivell de profunditat es mantenen al llarg de 69 capítols. Un humà sol esgotar-se o distreure's.

El que la IA NO fa bé

Aquí ve la part important, i honesta:

- **No tinc experiència viscuda.** Puc descriure com configurar Tailscale, però no puc explicar què se sent quan t'estàs 30 minuts barallant-te amb un acl que no rutlla. Aquestes coses les afegia l'autor (Bernat) quan les trobava, marcades com a "■■■ Pendent de validar" o afegides a peu de capítol.
- **Invento dates, números i detalls que semblen precisos però no ho són.** Tots els preus, les mides, els temps d'instal·lació d'aquest llibre són estimacions raonables, no pas mesures. Si tu vals més o menys, és normal.
- **No he provat res a la teva Raspberry.** Cap dels 12 capítols del M7 ha estat executat per mi a la teva mquina. L'autor els anirà validant a casa, i quan trobi diferències, s'actualitzaran.
- **Tinc biaixos de coneixement.** Soc bo explicant tecnologies populars (Docker, MQTT, Grafana), però menys fi en temes molt nous o molt específics. Si una explicació et sembla superficial, probablement és per això.
- **Cometo errors en el codi.** El codi YAML, Python, bash que surt al llibre és correcte en el 95% dels casos, però el 5% restant té fallades (versions obsoletes, comes que no toca, imports que no existeixen). Validar sempre.

Les limitacions del model

MiniMax-M3:Cloud és bo, però té limitacions:

- **No té memòria entre sessions.** Cada vegada que l'autor obria una nova sessió, jo no recordava res del que havíem parlat abans. Tots els detalls del projecte (IPs, configuracions, decisions preses) estaven en fitxers que jo llegia cada vegada.
- **No tinc accés directe a la Raspberry.** Tots els exemples que dono són teòrics o basats en documentació general. L'autor els validava a mà.
- **El nom del model "M3" ve de la família MiniMax M-series.** Són models de propòsit general, no pas entrenats específicament per a sistemes. Per tant, de tant en tant, es comporten de maneres inesperades.
- **El "Cloud" al nom vol dir que les meves respostes viatgen al núvol.** Tots els prompts i respostes passen per servidors externs. Per tant, **no és una solució d'IA local** malgrat que el plantejament del BernatLab defensa la privadesa. Per a tasques sensibles (claus SSH, contrasenyes, configuracions crítiques), l'autor mai no ha posat informació sensible als prompts. La IA local amb Ollama i un model com `llama3.1` o `mistral` sí que seria totalment privada — i s'explica al M4.

Què passa quan trobis un error

Si trobes alguna cosa que no funciona, és un **bug de la IA**. La millor manera de corregir-lo és:

1. Documentar exactament què has provat i què ha fallat.
2. Enviar la correcció a l'autor (Bernat Mora).
3. Ell decideix si val la pena actualitzar el llibre.

Aquest és un llibre viu. No està gravat en pedra. Cada nova versió és una oportunitat per millorar.

Agraïments

A Bernat Mora, per la paciència infinita d'anar validant cada capítol a la seva Raspberry, per la cura de revisar cada secció, i per la voluntat de mantenir el llibre honest sobre els seus orígens.

A la comunitat de programari lliure, per totes les eines que fan servir tant aquest llibre com el BernatLab: Debian, Docker, Tailscale, MQTT, Grafana, InfluxDB, Node-RED, Ollama, i tantes altres.

A tu, lector o lectora, per arribar fins aquí. Si has llegit el pròleg, és perquè t'interessa no només el què, sinó també el com. Això és bon senyal.

Comencem.

Capitol 33 — Què és la IA local i per què a Hort Osona

*"L'any 2026, tenir un assistent intel·ligent a casa ja no és qüestió d'enviar les teves dades a una empresa de Califòrnia. Pots tenir el cervell al teu Mac i la saviesa al teu hort."**

33.1 Què és la IA local

IA local (també anomenada **IA on-premise** o **self-hosted AI**) és executar models d'intel·ligència artificial al teu propi ordinador, sense enviar res al núvol. Quan fas una pregunta a ChatGPT, el text viatja a un servidor d'OpenAI, es processa allà, i torna la resposta. Amb IA local, tot el procés passa dins la teva màquina.

Això és possible gràcies a dues coses:

1. **Models petits però capaços.** Fa tres anys, per tenir un model útil calia un servidor amb 100 GB de RAM. Avui, n'hi ha prou amb un MacBook amb 16 GB per fer córrer un model que escriu en català, raona sobre text, i segueix instruccions complexes.

1. **Eines com Ollama.** Ollama és una aplicació que descarrega models, els posa en marxa amb una sola comanda, i exposa una API local compatible amb la d'OpenAI. Tu li dius `ollama run gemma3:12b` i tens un assistent a la terminal.

33.2 Diferència entre IA al núvol i IA local

Aspecte	IA al núvol (ChatGPT, Claude)	IA local (Ollama)
On es processa	Servidors de l'empresa	El teu Mac/RPi
Privadesa	Les dades surten del teu ordinador	Tot es queda a casa
Cost	Subscripció mensual (~20 €/mes)	Gratuït, després de comprar el hardware
Velocitat	Ràpida (cloud potent)	Depèn del model i el hardware
Català	Molt bo	Variable, depèn del model
Disponibilitat	Si tens Internet	Funciona offline
Personalització	Limitada	Pots entrenar amb les teves dades

Per a Hort Osona, **la IA local és ideal** perquè:

- Les teves 76 fitxes de cultius, 30+ guies, i el coneixement de l'hort no surten de casa.
- Pots preguntar coses específiques del teu terreny, varietats locals, el clima d'Osona.
- No pagues subscripció.
- Funciona encara que TallCable o el router estiguin caiguts.

33.3 Què pots fer amb IA local a Hort Osona

Aquestes són les aplicacions reals que veurem al llarg del mòdul:

1. **Assistent hort Osona.** Li preguntes en català: "Quan he de sembrar les carbasses a la zona d'Osona?" Et contesta basant-se en les 76 fitxes i el calendari lunar d'Osona.

1. **Resum de documents.** Tens un PDF de 50 pàgines sobre compostatge. Li dius "Fes-me'n un resum d'una pàgina" i te'l fa.

1. **Cercar informació.** Tens 30+ guies. En lloc de cercar per paraules clau, li preguntes "Què he de fer si el tomàquet té míldiu?" i troba els paràgrafs rellevants.

1. **Generar textos.** Necessites un correu al veí per demanar permís d'ampliar l'hort? Li dius "Escriu un correu amable" i te l'escriu.

1. **Veü.** Li parles en lloc d'escriure, mentre treballes a l'hort. Et contesta amb veü.

1. **Integració amb el BernatLab.** La Raspberry consulta Ollama al Mac via Tailscale. Les dades dels sensors LoRa alimenten un assistent que t'avisa: "El sòl del sector 3 està massa sec; rega'l demà al matí."

33.4 Què NO és (i què sí)

NO és:

- Una còpia exacta de ChatGPT. Els models locals són menys capaços que GPT-4 o Claude Opus. Són útils però no miracles.
- Màgia. Necessites bons models, bons prompts, i bons documents.
- Privat per defecte. Si copies les dades a un model al núvol, sí que surten. Cal configurar bé.

Sí és:

- Prou bona per a 90% de les tasques quotidianes.
- Molt personalitzable. Pots afinar-la amb els teus documents.
- Transparent. Saps exactament què fa perquè és al teu ordinador.
- Econòmica. Després del hardware, tot és gratis.

33.5 Com aprendràs a usar-la

El mòdul segueix un camí pràctic:

1. **Cap 34:** instal·lar Ollama al Mac i fer les primeres preguntes.
2. **Cap 35:** com triar el model adequat (català, mida, velocitat).
3. **Cap 36:** com funciona la cerca semàntica (embeddings, vectorstores).
4. **Cap 37:** muntar un RAG amb les 76 fitxes d'hort.
5. **Cap 38:** client web per parlar amb l'assistent des del navegador.

6. **Cap 39:** integrar Ollama amb l'API del BernatLab.
7. **Cap 40:** afegir-hi veu.
8. **Cap 41:** privadesa i seguretat.
9. **Cap 42:** casos d'ús reals, amb 10 consultes que pots fer ja.

33.6 El que necessites

Hardware mínim:

- Mac amb Apple Silicon (M1, M2, M3, M4) i 16 GB de RAM. Recomanable 32 GB per a models grans.
- Alternativa: PC amb targeta gràfica NVIDIA (RTX 3060 o superior) amb 8+ GB VRAM.
- Raspberry Pi 4 amb 8 GB: només per a models molt petits (1B-3B paràmetres).

Software:

- macOS, Linux o Windows 10/11.
- 10-30 GB d'espai lliure per als models.
- Ollama (gratis, descarregable de ollama.com).

Coneixements previs:

- Ús bàsic de la terminal (Cap 3 del Mòdul 1).
- Python intermedi (saber instal·lar llibreries, escriure scripts petits).

33.7 Privadesa des del primer dia

Una regla d'or:

Si una dada és prou sensible per no explicar-la a un desconegut al carrer, no l'enviïs a cap servei d'IA al núvol.

Això inclou:

- Dades mèdiques.
- Informació financera personal.
- Correspondència privada.
- Dades d'altres persones sense el seu consentiment.
- Secrets comercials o professionals.

Amb IA local, aquest problema desapareix: la dada no surt del teu ordinador. Però encara cal configurar-ho bé, cosa que veurem al Cap 41.

33.8 El futur a Hort Osona

A mesura que escrigui més capítols del llibre i tinguis més dades, l'assistent es fa més savi:

- **Fitxes de cultius** (76) → sap tot sobre cada varietat.
- **Guies** (30+) → sap sobre compost, plagues, associacions, calendari lunar.
- **Dades de sensors** (M3) → sap l'estat real del teu hort en temps real.
- **Bitàcoles** → aprèn què funciona i què no al teu terreny.
- **Correspondència** (si vols) → recorda el que has après amb cada veí.

D'aquí un any, el teu Hort Osona serà un dels millor documentats de Catalunya, i el teu assistent podrà respondre qualsevol pregunta que li facis sobre el que has après.

33.9 Resum

La IA local et permet tenir un assistent potent al teu Mac o Raspberry, sense enviar dades al núvol, sense subscripció, i personalitzable amb els teus propis documents. Per a Hort Osona és una eina natural: 76 fitxes de cultius, 30+ guies, dades de sensors, i tot el coneixement que has anat acumulant. En els propers capítols aprendrem a instal·lar Ollama, triar un bon model, i muntar un sistema RAG complet.

33.10 Exercicis pràctics

1. Comprova quin Mac tens: Apple Icon → About This Mac. Anota el xip (M1/M2/M3/M4) i la RAM.
2. Comprova l'espai lliure al disc: `df -h` (a la terminal). Hauries de tenir almenys 30 GB.
3. Comprova la versió de macOS: Apple Icon → About This Mac. Hauries de tenir macOS 13+ per a la millor compatibilitat.
4. Fes una llista de les 5 preguntes que t'agradaria poder fer al teu assistent hort Osona.
5. Comprova que tens Python 3.10+: `python3 --version`.
6. Documenta al README del projecte el teu hardware i les preguntes que tens en ment.

Paraules clau: **IA local, self-hosted AI, on-premise, Ollama, RAG, embeddings, vectorstore, LLM, model, prompt, privadesa, Apple Silicon, M1, M2, M3, M4, Mac, RAM, 16 GB, 32 GB, català, model de llengua, self-hosted, núvol, subscripció, ChatGPT, Claude, GPT-4, Opus, alternativa, open source, codi obert, transparent.**

Capítol 34 — Ollama al Mac: instal·lació i primera conversa

"Instal·lar Ollama al Mac és com obrir una ampolla de vi: cinc minuts, i ja tens alguna cosa valuosa a la mà."

34.1 Què és Ollama

Ollama (ollama.com) és una aplicació gratuïta i de codi obert que permet executar models d'IA localment amb una sola comanda. Està disponible per a macOS, Linux i Windows. La va crear Jeffrey Morgan i l'equip d'Ollama, i s'ha convertit en l'estàndard de facto per a IA local.

Ollama fa quatre coses:

1. **Descarrega models** d'un registre central (similar a Docker Hub, però per a models).
2. **Executa models** al teu hardware, optimitzant per a Apple Silicon (Metal), NVIDIA (CUDA) o CPU.
3. **Serveix una API local** (a <http://localhost:11434>) compatible amb la d'OpenAI.
4. **Gestiona la memòria** alliberant RAM quan no uses el model.

34.2 Instal·lació al Mac

Mètode 1: descarregar de la web (recomanat)

1. Vés a <https://ollama.com/download/mac>.
2. Descarrega el .dmg.
3. Obre'l i arrossega Ollama a la carpeta Aplicacions.
4. Executa Ollama. Apareixerà una icona de camell a la barra de menú.

Mètode 2: Homebrew (si el tens)

```
brew install ollama
```

Mètode 3: comanda directa

```
curl -fsSL https://ollama.com/install.sh | sh
```

Aquest mètode funciona a macOS, Linux i WSL. A Windows, descarrega l'instal·lador de la web.

34.3 Verificar la instal·lació

Obre un terminal i comprova:

```
ollama --version
```

Hauries de veure alguna cosa com `ollama version 0.5.7` (o superior). Si no, afegeix Ollama al PATH:

```
# Si usaves Homebrew:  
export PATH="/usr/local/bin:$PATH"
```

```
# Si vas instal·lar manualment, crea un enllaç simbòlic:
sudo ln -s /Applications/Ollama.app/Contents/Resources/ollama /usr/local/bin/ollama
```

34.4 Descarregar el primer model

El primer pas és descarregar un model. Ollama ofereix molts. Per començar, recomano **gemma3:4b** (Google, 4 mil milions de paràmetres, ~3 GB):

```
ollama pull gemma3:4b
```

Això descarrega el model. Pot trigar entre 2 i 10 minuts segons la teva connexió.

Quan acabi, verifica:

```
ollama list
```

Hauries de veure:

NAME	ID	SIZE	MODIFIED
gemma3:4b	...	3.1 GB	2 minutes ago

34.5 Primera conversa

Ara pots parlar amb el model. Prova:

```
ollama run gemma3:4b "Hola, qui ets?"
```

Hauries de rebre una resposta en qüestió de segons. Si tot funciona, ja tens IA local al Mac.

Per sortir, escriu `/bye` o prem `Ctrl+D`.

34.6 Comandes bàsiques d'Ollama

Comanda	Què fa
<code>ollama pull <model></code>	Descarrega un model
<code>ollama run <model></code>	Inicia una conversa
<code>ollama list</code>	Llista els models descarregats
<code>ollama rm <model></code>	Esborra un model
<code>ollama ps</code>	Mostra els models que s'estan executant
<code>ollama show <model></code>	Mostra els detalls d'un model
<code>ollama cp <src> <dst></code>	Copia un model amb un altre nom
<code>ollama stop <model></code>	Atura un model en execució

34.7 Modes d'ús

Mode conversa (REPL)

Quan executes `ollama run <model>`, entres en mode conversa. Pots escriure qualsevol cosa i el model et respon. Per mantenir el context (que el model recordi el que has dit), continua en la mateixa sessió.

Per començar una conversa nova, obre una altra finestra de terminal.

Mode script (un sol prompt)

```
ollama run gemma3:4b "Explica'm què és un embedding en 3 frases"
```

Això envia el prompt, rep la resposta, i surt.

Mode API

Ollama escolta a `http://localhost:11434` quan està en marxa. Pots enviar peticions HTTP:

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3:4b",
  "prompt": "Què és un cogombre?",
  "stream": false
}'
```

Això retorna JSON amb la resposta. Veurem com integrar-ho al Cap 39.

34.8 Com fer bones preguntes

La clau per treure profit d'un model local és **fer bones preguntes**. Algunes pautes:

1. **Sigues específic.** "Explica el compostatge" és vague. "Explica el compostatge en climes humits amb palla i restes de cuina" és millor.
1. **Dona context.** "Sóc a Osona, tinc un hort de 200 m² amb tomàquets i carbasses. Quina varietat de carbassa em recomanes?" és millor que "Quina carbassa plantar?".
1. **Demana format concret.** "Fes-me'n una llista de 5 punts" o "Explica-m'ho en 3 paràgrafs".
1. **Demana verificació.** "Si no saps la resposta, digue-m'ho. No inventis informació." Els models tendeixen a inventar-se coses (les anomenem "al·lucinacions").
1. **Posa exemples.** "Vull un correu com aquest: 'Benvolgut veí, ...'". El model imitarà l'estil.

34.9 Configuració avançada: system prompt

Quan executes un model, Ollama permet passar un **system prompt** (instruccions de sistema, missatges inicials que condicionen tot el comportament del model) que canvia el comportament. Per exemple:

```
ollama run gemma3:4b "
Ets un expert en horticultura ecològica a la comarca d'Osona.
Respon sempre en català.
Sigues pràctic i directe, sense floritures.
Si no saps una resposta concreta, recomana consultar les fitxes locals.
"
```

El model respondrà seguint aquestes instruccions fins que tanquis la sessió.

34.10 Persistència de la configuració: Modelfiles

Si vols que el system prompt estigui sempre disponible, pots crear un **Modelfile**:

```
# Crea un fitxer anomenat Modelfile-hort
cat > Modelfile-hort << 'EOF'
```

```
FROM gemma3:4b
SYSTEM ""
Ets l'assistent Hort Osona. Coneixes les 76 fitxes de cultius del projecte
BernatLab. Respon sempre en català. Sigues pràctic. Si no saps una resposta,
digues "Consulta la fitxa corresponent a Hort Osona" i no inventis.
""
PARAMETER temperature 0.3
PARAMETER num_ctx 4096
EOF

# Crea un model personalitzat
ollama create hort-osona -f Modelfile-hort

# Usa'l
ollama run hort-osona "Quan he de sembrar tomàquets?"
```

Això és molt potent: pots tenir múltiples "personalitats" del mateix model base, cadascuna optimitzada per una tasca.

34.11 El primer model: què esperar

Amb gemma3:4b (4B paràmetres, ~3 GB):

- Velocitat: 20-50 tokens/segon en un Mac M1 amb 16 GB.
- Català: acceptable, no perfecte.
- Raonament: limitat, però útil per a resums i consultes senzilles.
- Memòria: 3-4 GB de RAM ocupats.

Per a tasques més complexes (raonament llarg, codi, multilingüe avançat), necessitaràs models més grans. Això ho veurem al Cap 35.

34.12 Primers problemes i solucions

Problema 1: el model no descarrega.

Comprova la connexió: `curl -I https://ollama.com`. Si funciona, mira l'espai lliure: `df -h`. Necessites almenys 5 GB.

Problema 2: el model respon en anglès.

El model és multilingüe, però tendeix a l'anglès. Força el català amb un system prompt explícit: "Respon SEMPRES en català, encara que et preguntin en un altre idioma."

Problema 3: el Mac s'escalfa molt.

Els models consumeixen molta CPU/GPU. És normal. Si et molesta, redueix el context (paràmetre `num_ctx`) o usa un model més petit.

Problema 4: la resposta és molt lenta.

Si tens molts programes oberts, tanca'ls. Si el model és massa gran per al teu Mac, descarrega'n un de més petit (gemma3:2b o phi3:mini).

Problema 5: "address already in use".

El port 11434 està ocupat. Probablement tens una altra instància d'Ollama corrent. Tanca-la i reobre.

34.13 Compartir el Mac amb la Raspberry

Si vols que la Raspberry accedeixi a Ollama al Mac, cal fer dues coses:

1. **Permetre connexions externes.** Per defecte, Ollama només escolta a `localhost`. Per canviar-ho:

```
# Atura Ollama
pkill ollama

# Inicia'l escoltant a totes les interfícies
OLLAMA_HOST=0.0.0.0:11434 ollama serve &
```

1. **Assegurar que la Raspberry pot arribar al Mac.** Si tens Tailscale configurat, simplement:

```
# Des de la Raspberry
curl http://<mac-tailscale-ip>:11434/api/tags
```

Si funciona, veuràs la llista de models.

Per a ús continu, pots crear un servei launchd (gestor de serveis automàtic de macOS) que mantingui Ollama en marxa i escoltant a la xarxa Tailscale.

34.14 Resum

Hem après a instal·lar Ollama al Mac, descarregar el primer model (gemma3:4b), mantenir una conversa, configurar system prompts i Modelfiles, i compartir Ollama amb la Raspberry via Tailscale. Al proper capítol veurem com triar el millor model per a les nostres necessitats: mida, velocitat, qualitat, i especialment el català.

34.15 Exercicis pràctics

1. Instal·la Ollama al Mac seguint els passos.
2. Descarrega `gemma3:4b` i `gemma3:12b` (si tens RAM).
3. Fes-li 5 preguntes relacionades amb l'hort i avaluja la qualitat de les respostes.
4. Crea un Modelfile anomenat `hort-osona` amb un system prompt útil.
5. Configura Ollama per escoltar a `0.0.0.0:11434` i comprova que la Raspberry pot accedir-hi.
6. Fes una llista de 5 tasques que voldries que l'assistent pogués fer.
7. Documenta al README el model que uses, el Modelfile, i les primeres impressions.

Paraules clau: **Ollama**, **install**, **descarregar**, **model**, **gemma3**, **phi3**, **llama**, **mistral**, **terminal**, **REPL**, **API**, **system prompt**, **Modelfile**, **temperature**, **num_ctx**, **paràmetres**, **català**, **Tailscale**, **0.0.0.0**, **11434**, **curl**, **JSON**, **streaming**, **Mac**, **Apple Silicon**, **M1**, **M2**, **M3**, **M4**, **Metal**, **MLX**, **launchd**, **dimonis**, **daemon**, **xarxa local**, **pkill**, **kill**, **process**, **foreground**, **background**, **model base**, **custom model**, **etiqueta**, **tag**, **registre**, **registry**, **Docker Hub**, **anàleg**, **semàntica**, **token**, **tokenització**, **vocabulari**, **context window**, **max tokens**, **longitud màxima**, **prompt**, **response**, **generation**, **sampling**, **top-k**, **top-p**, **temperature**, **seed**, **determinisme**, **randomness**, **creatividad**, **precisió**, **al·lucinació**, **inventar**, **verificar**, **fact-checking**, **font fiable**, **citació**, **referència**.

Capitol 35 — Com triar el millor model: mida, velocitat, qualitat, català

"Triar un model d'IA és com triar una eina: la clau no és la que té més prestacions, sinó la que s'adapta a la feina."

35.1 Què mesura un "bon" model

Hi ha quatre dimensions que importen:

- 1. **Mida** (quantitat de paràmetres). Més paràmetres = més savi però més lent i pesat. Es mesura en mil milions de paràmetres (B). Exemples: 1B, 3B, 7B, 13B, 70B.
- 1. **Velocitat** (tokens per segon). Quants tokens pot generar per segon. Un Mac M1 amb un model 7B genera ~20-30 tokens/s. Un model 70B genera 2-5 tokens/s.
- 1. **Qualitat** (precisió, raonament, coneixement). En general, com més gran el model, millor la qualitat. Però hi ha models petits que són excel·lents en tasques específiques.
- 1. **Català** (suport lingüístic). Alguns models parlen millor català que d'altres. Cal validar amb proves reals.

35.2 Models recomanats el 2026

Aquesta llista canvia cada mes. Però a data de juliol 2026, els models locals més interessants per al BernatLab són:

Models petits (1B-4B) — per a Mac amb 8-16 GB de RAM

Model	Mida	Qualitat	Català	Ús recomanat
gemma3:2b	1.6 GB	Bona per a tasques simples	Aceptable	Comandes ràpides, resums curts
gemma3:4b	3.1 GB	Bona	Bona	Primera opció per a 8-16 GB
phi3:mini	2.3 GB	Excel·lent per a la mida	Acceptable	Raonament, codi
llama3.2:3b	2.0 GB	Bona	Bona	Alternativa a gemma3
qwen2.5:3b	1.9 GB	Bona	Molt bona	Multilingüe

Models mitjans (7B-14B) — per a Mac amb 24-32 GB de RAM

Model	Mida	Qualitat	Català	Ús recomanat
gemma3:12b	8.1 GB	Molt bona	Molt bona	Recomanat per a 16-32 GB
llama3.2:11b	6.9 GB	Molt bona	Bona	Alternativa
mistral:7b	4.1 GB	Bona	Bona	Clàssic
mixtral:8x7b	26 GB	Excel·lent	Bona	Si tens 32 GB
qwen2.5:14b	8.7 GB	Molt bona	Excel·lent	El millor en català

Models grans (30B-70B) — per a Mac amb 64+ GB o PC amb GPU

Model	Mida	Qualitat	Català	Ús recomanat
llama3.1:70b	40 GB	Excel·lent	Molt bona	Si tens molta RAM
gemma3:27b	17 GB	Excel·lent	Excel·lent	Bon equilibri
qwen2.5:32b	19 GB	Excel·lent	Excel·lent	El millor en català
deepseek-r1:32b	19 GB	Excel·lent	Bona	Raonament complex

35.3 Com triar segons el teu hardware

MacBook Air M1/M2 amb 8 GB

- Limita't a models d'1-3B.
- Recomanació: `gemma3:2b` o `phi3:mini`.
- Compromís: qualitat baixa però usable per a consultes curtes.

MacBook Pro M2/M3 amb 16 GB

- Models de 4-8B funcionen bé.
- Recomanació: `gemma3:4b` per defecte, `llama3.2:11b` si necessites més.
- Bon equilibri velocitat/qualitat.

MacBook Pro M3 Max/M4 amb 32-64 GB

- Models de 14-27B funcionen bé.
- Recomanació: `gemma3:12b` o `qwen2.5:14b` per defecte.
- Qualitat alta, velocitat acceptable.

Mac Studio M2 Ultra amb 64-192 GB

- Models de 30-70B funcionen.

- Recomanació: `gemma3:27b` o `llama3.1:70b`.
- Qualitat màxima.

Raspberry Pi 4 amb 4-8 GB

- Només models d'1-2B.
- Recomanació: `gemma3:2b` o `phi3:mini`.
- Útil només per a comandes molt senzilles.

PC amb GPU NVIDIA RTX 3060+ (8 GB VRAM)

- Models de 7-13B amb quantització (tècnica que redueix la mida del model comprimint els pesos en menys bits, amb pèrdua mínima de qualitat).
- Recomanació: `mistral:7b-q4` (q4 vol dir quantització de 4 bits).
- Molt bona velocitat si tens CUDA ben configurat.

35.4 Què és la quantització

Els models venen en diverses mides segons la quantització. Per exemple:

- `gemma3:12b` (8.1 GB) — versió completa, 16 bits per paràmetre.
- `gemma3:12b-q4_K_M` (5.5 GB) — quantitzat a 4 bits, qualitat gairebé idèntica.
- `gemma3:12b-q8_0` (7.5 GB) — quantitzat a 8 bits, qualitat excel·lent.

La regla és senzilla: **amb la quantització Q4_K_M tens el millor equilibri mida/qualitat**. Q8 és lleugerament millor però ocupa més. Q2 i Q3 són massa agressives (perden qualitat notable).

Per descarregar una versió quantitzada:

```
ollama pull gemma3:12b-q4_K_M
```

35.5 Com avaluar el català

El català varia molt entre models. Comprovacions pràctiques:

1. **Cultura general.** Pregunta: "Qui va escriure 'La plaça del Diamant'?" Si respon "Mercè Rodoreda", perfecte.
1. **Varietats dialectals.** Pregunta: "Com es diu 'mongeta' a Osona?" Si respon amb termes locals, encara millor.
1. **Termes tècnics.** Pregunta: "Què és el mildiu del tomàquet?" Si dóna una resposta correcta i completa, endavant.
1. **Generació de text.** Demana-li que escrigui un correu o una recepta. Mira si l'estil és natural.
1. **Codi.** Si vols que t'ajudi amb Python o scripts, avalua la qualitat del codi.

Fes una llista de 10 preguntes representatives i avalua cada model. Al cap de 2-3 dies tindràs clar quin és el millor per a tu.

35.6 Com canviar de model fàcilment

Pots tenir molts models descarregats alhora i canviar segons la tasca:

```
# Llista els que tens
ollama list

# Per defecte usa el millor
ollama run qwen2.5:14b "..."/>

```

Si vols que el teu sistema sempre usi un model concret per defecte, crea un alias:

```
# Crea un alias "assistent" apuntant al millor model
ollama cp gemma3:12b assistent

# Ara pots fer
ollama run assistent "Hola"
```

Això és útil quan configures l'API al Cap 39: simplement fas servir el nom de l'alias.

35.7 El cas especial: el català

El català és un idioma amb menys recursos que l'anglès o el castellà, però la situació ha millorat molt. Al 2026, els millors models en català són:

- **qwen2.5** (Alibaba): entrenat explícitament en molts idiomes, català inclòs.
- **gemma3** (Google): bon català, sobretot en les versions 12B+.
- **llama3.1** (Meta): bon català en versions grans.

Els pitjors en català solen ser:

- Models antics (llama2, mistral v1).
- Models massa petits (<3B) de qualsevol família.
- Models entrenats només en anglès.

Si el català és crític (i per a tu ho és), tria `qwen2.5:14b` o `gemma3:12b-q4_K_M`. Són els millors equilibris.

35.8 Com optimitzar la velocitat

Algunes tècniques per accelerar la resposta:

1. **Reduir el context.** Per defecte, Ollama carrega 2048 tokens. Si la teva consulta és curta, pots posar `num_ctx 1024`.
1. **Usar quantització Q4_K_M.** És el punt òptim.
1. **Tancar altres aplicacions.** El Mac comparteix RAM entre totes les aplicacions. Si tens Chrome amb 50 pestanyes, l'Ollama va més lent.
1. **Pre-escalfar el model.** La primera resposta és lenta perquè carrega el model a RAM. Després va ràpid.

1. **Servir vs conversar.** Si vols velocitat màxima, usa l'API HTTP (Cap 39) en lloc del mode conversa.

35.9 Les meves recomanacions finals

Per al teu cas (Mac, Osona, hort, català):

- **Si tens 16 GB:** comença amb `gemma3:4b` per explorar, puja a `gemma3:12b-q4_K_M` quan vulguis més qualitat.
- **Si tens 32 GB:** directament `gemma3:12b` o `qwen2.5:14b`.
- **Si tens 64 GB:** `qwen2.5:32b` per a la màxima qualitat en català.
- **Per a resums ràpids:** `gemma3:4b` sempre va bé.
- **Per a tasques complexes:** puja a 12B o 14B.

35.10 Prova pràctica: 3 models, 5 preguntes

Per comparar models, fes aquesta prova:

```
# Descarrega 3 models
ollama pull gemma3:4b
ollama pull gemma3:12b
ollama pull qwen2.5:14b

# Crea un script de proves
cat > test-models.sh << 'EOF'
#!/bin/bash
MODELS=("gemma3:4b" "gemma3:12b" "qwen2.5:14b")
QUESTIONS=(
  "Quan he de sembrar carbasses a la comarca d'Osona?"
  "Explica'm què és el mildiu del tomàquet i com tractar-lo de forma ecològica"
  "Fes-me'n un resum de 5 línies sobre el calendari lunar aplicat a l'hort"
  "Quines associacions de cultius són bones per a un hort petit de 100 m²?"
  "Com puc fer compost amb restes de cuina si visc en un pis?"
)
for m in "${MODELS[@]}; do
  echo "===== $m ====="
  for q in "${QUESTIONS[@]}; do
    echo "Q: $q"
    ollama run "$m" "$q" 2>/dev/null | head -10
    echo "---"
  done
done
EOF

chmod +x test-models.sh
./test-models.sh > resultats.txt
```

Llegeix els resultats i tria el model que t'agradi més. No hi ha una resposta única — depèn de les teves prioritats.

35.11 Quan actualitzar el model

Els models s'actualitzen sovint. Bones pràctiques:

- **Cada 3-6 mesos**, mira si n'hi ha un de millor.
- **Descarrega'l i prova'l** sense eliminar l'antic.
- **Compara** amb les teves 5-10 preguntes representatives.
- **Substitueix** quan el nou sigui clarament millor.

35.12 Resum

Hem après a triar un model segons el hardware, la mida, la velocitat, i el català. Hem vist els millors models del 2026, com quantitzar, com avaluar la qualitat, i hem donat una recomanació concreta per al teu cas. Al proper capítol veurem com funcionen els embeddings i les bases vectorials, la base tècnica del RAG que muntarem al Cap 37.

35.13 Exercicis pràctics

1. Comprova el teu hardware (xip, RAM).
2. Descarrega 2-3 models recomanats per al teu hardware.
3. Fes la prova pràctica de 5 preguntes i avalua les respostes.
4. Crea un Modelfile "assistent-hort" amb el millor model.
5. Documenta al README quin model has triat i per què.
6. Compara el rendiment amb i sense quantització Q4.
7. Comparteix les teves impressions al README del projecte.

Paraules clau: **model, paràmetres, B, mil milions, tokens, tokens per segon, velocitat, qualitat, català, gemma3, qwen2.5, llama3, mistral, mixtral, deepseek, phi3, quantització, Q4_K_M, Q8, 16 bits, 4 bits, 8 bits, GGUF, GPTQ, AWQ, MLX, Apple Silicon, Metal, CUDA, VRAM, RAM, context, num_ctx, temperature, sampling, top-k, top-p, determinisme, seed, MODelfile, alias, registre, registry, benchmark, MMLU, IFEval, MT-Bench, AReQA, pàgina d'avaluació, leaderboard, Open LLM Leaderboard, LMSYS, Hugging Face, comparació, decisió, prova pràctica, benchmark personalitzat, actualització, manteniment.**

Capítol 36 — Embeddings i bases vectorials

""Si cerques per paraules exactes, trobaràs documents amb aquestes paraules. Si cerques per significat, trobaràs documents que parlen del que vols saber, encara que no usin les mateixes paraules.""

36.1 El problema: la cerca per paraules clau no és prou

Imagina que tens les 76 fitxes d'hort i un dia vols saber **"què fer quan els tomàquets tenen taques grogues a les fulles"**. Si cerques a mà:

- "tomaques taca groga fulla" — pot ser que trobis poques coses perquè el text pot dir "fulles groguenques", "clorosi", "deficiència de nitrogen", etc.
- Necessaries llegir cada fitxa, una per una.

El que voldries és: "troba'm els paràgrafs que parlen d'això, encara que no usin les mateixes paraules". Això és el que fan els **embeddings** i les **bases vectorials**.

36.2 Què és un embedding

Un **embedding** (en català, **representació vectorial** o **incrustació**) és una manera de convertir text en una llista de números (un vector) que captura el **significat** del text.

Per exemple:

- "El gos borda" → [0.12, -0.34, 0.78, ..., 0.45] (384 o 1024 o 4096 números)
- "El ca lladra" → [0.11, -0.33, 0.79, ..., 0.46] (molt similar!)
- "Avui plou" → [0.87, 0.23, -0.45, ..., -0.12] (molt diferent)

Això és màgia matemàtica: frases amb significat similar tenen vectors propers. Frases amb significat diferent tenen vectors allunyats.

Aquesta propietat es mesura amb la **similaritat del cosinus** (cosine similarity, mètrica que mesura l'angle entre dos vectors: 1.0 = idèntic, 0.0 = orthogonal, -1.0 = oposat). Dos textos que parlen del mateix tindran una similaritat alta (>0.8). Dos textos no relacionats tindran una similaritat baixa (<0.3).

36.3 Com es crea un embedding

Hi ha models entrenats específicament per a aquesta tasca. Els més coneguts:

- **OpenAI text-embedding-3-small/large** (al núvol, pagament).
- **BGE** (BAAI, gratuït, molt bo).
- **E5** (Microsoft, gratuït, multilingüe).
- **mE5** (multilingüe, inclou català).
- **Nomic Embed** (obert, bona qualitat).
- **Ollama** (sí, Ollama també pot generar embeddings!).

A Ollama:

```
# Descarrega un model d'embeddings
ollama pull nomic-embed-text

# Genera l'embedding d'un text
curl http://localhost:11434/api/embeddings -d '{
  "model": "nomic-embed-text",
  "prompt": "El gos borda fort"
}'
```

Això retorna un vector de 768 números. Cadascun captura algun aspecte del significat.

36.4 Què és una base vectorial

Una **base vectorial** (vector store, vector database) és un sistema que:

1. **Emmagatzema** milions de vectors associats a fragments de text.
2. **Cerca ràpidament** els vectors més similars a una consulta.

Les bases vectorials més conegudes:

- **ChromaDB** — la més fàcil d'usar, ideal per començar.
- **FAISS** (Facebook) — molt ràpida, per a molts vectors.
- **LanceDB** — moderna, eficient, bona per a hobby.
- **Qdrant** — servidor complet, escalable.
- **Milvus** — solucions professionals, molt ràpid.
- **Pinecone** — al núvol, pagament.

Per al BernatLab, recomano **ChromaDB** o **LanceDB** per simplicitat. Si volem alguna cosa més professional, **Qdrant**.

36.5 ChromaDB: el primer pas

ChromaDB és la base vectorial més fàcil d'utilitzar. S'instal·la amb pip i funciona localment.

Instal·lació

```
pip install chromadb
```

Ús bàsic en Python

```
import chromadb

# Inicialitzar (per defecte emmagatzema a ~/.chromadb)
client = chromadb.PersistentClient(path="./vectorstore")

# Crear o obtenir una col·lecció
collection = client.get_or_create_collection(name="hort_osona")

# Afegir documents
collection.add(
    documents=[
        "El tomàquet de Penedès prefereix sòls ben drenats i assolellats.",
        "Les carbasses necessiten molt espai, almenys 2 m² per planta.",
        "El mildiu és un fong que apareix amb humitat alta."
    ],
    metadatas=[
        {"source": "fitxa-tomaque.md", "tema": "tomàquet"},
        {"source": "fitxa-carbassa.md", "tema": "carbassa"},
        {"source": "guia-plagues.md", "tema": "malalties"}
    ],
    ids=["doc1", "doc2", "doc3"]
)

# Cercar per similitud
results = collection.query(
    query_texts=["Quan he de plantar carbasses?"],
    n_results=2
)

# Mostrar els resultats
for doc, meta in zip(results['documents'][0], results['metadatas'][0]):
    print(f" {meta['source']}: {doc}")
```

La cerca retorna els 2 documents més similars a la pregunta, ordenats per rellevància.

36.6 Com funciona la cerca per similitud

Quan fas una consulta:

1. **Converteix la consulta en un vector** usant el mateix model d'embeddings.
2. **Calcula la distància** entre el vector de la consulta i tots els vectors emmagatzemats.
3. **Retorna els k més propers** (k=2 a l'exemple anterior).

Això és extremadament ràpid: per a 10.000 vectors, la cerca triga menys d'un segon.

36.7 Com fragmentar els documents

Un embedding té una mida màxima (normalment 512-2048 tokens, és a dir, unitats de text que el model processa, aproximadament 1 token = 0.75 paraules en català). Si tens un document de 5.000 tokens, no pots fer-ne un sol embedding. Cal **fragmentar** (chunking) en parts més petites.

Estratègies comunes de fragmentació:

1. **Per mida fixa.** Trossos de 500 tokens amb solapament de 50. Senzill i efectiu.
1. **Per paràgrafs o seccions.** Cada secció del document és un chunk. Mantén la coherència.

1. **Semàntica.** Parteix on hi ha canvis de tema. Més sofisticat.

1. **Per estructures Markdown.** Capçaleres, llistes, blocs de codi. Molt adequat per a llibres tècnics.

Per al BernatLab, recomano **per mida fixa (500 tokens) amb solapament (50 tokens)**. Senzill i funciona bé.

36.8 Ús pràctic: indexar les 76 fitxes d'hort

Un script per indexar totes les fitxes:

```
#!/usr/bin/env python3
"""
index_hort_osona.py
Indexa tots els fitxers Markdown de Hort Osona a ChromaDB.
"""

import os
from pathlib import Path
import chromadb
from chromadb.utils import embedding_functions

# Configuració
HORT_DIR = Path("/home/bernat/bernatlab/projects/hort-osona")
VECTORSTORE_DIR = "./vectorstore"
COLLECTION_NAME = "hort_osona"

# Model d'embeddings – cal que estigui a Ollama
ollama_ef = embedding_functions.OllamaEmbeddingFunction(
    model_name="nomic-embed-text",
    url="http://localhost:11434/api/embeddings"
)

# Inicialitzar ChromaDB
client = chromadb.PersistentClient(path=VECTORSTORE_DIR)
collection = client.get_or_create_collection(
    name=COLLECTION_NAME,
    embedding_function=ollama_ef
)

# Funció per fragmentar un document
def fragmentar(text, mida=500, solapament=50):
    """Fragmenta text en trossos amb solapament."""
    paraules = text.split()
    fragments = []
    i = 0
    while i < len(paraules):
        fragment = " ".join(paraules[i:i+mida])
        fragments.append(fragment)
        i += mida - solapament
    return fragments

# Indexar tots els .md
idx = 0
for md_file in HORT_DIR.rglob("*.md"):
    if md_file.name.startswith("."):
        continue
    text = md_file.read_text(encoding="utf-8")
    fragments = fragmentar(text)
    for n, frag in enumerate(fragments):
        collection.add(
            documents=[frag],
```



```

        metadatas=[{
            "source": str(md_file.relative_to(HORT_DIR)),
            "fragment": n,
            "tema": md_file.stem
        }],
        ids=[f"{md_file.stem}-{n}"]
    )
    idx += 1
    print(f"  Indexat: {md_file.name} ({len(fragments)} fragments)")

print(f"\nTotal: {idx} fragments indexats a ChromaDB")

```

Executa'l una vegada, i ja tens les 76 fitxes (i totes les guies) en una base vectorial.

36.9 Com escollir el model d'embeddings

No tots els models d'embeddings són igual de bons. Al 2026, els millors per a text multilingüe (inclòs català) són:

Model	Mida	Qualitat	Català	Velocitat
nomic-embed-text	274 MB	Excel·lent	Bona	Molt ràpid
mxbai-embed-large	670 MB	Excel·lent	Bona	Ràpid
bge-m3	2.3 GB	Excel·lent	Molt bona	Mitjà
multilingual-e5-large	2.2 GB	Excel·lent	Molt bona	Mitjà
snowflake-arctic-embed-2	2 GB	Molt bona	Bona	Ràpid

Recomanació per al BernatLab: **nomic-embed-text** per defecte. Si vols més qualitat, **bge-m3** o **multilingual-e5-large**.

36.10 Com avaluar la qualitat de la cerca

Per validar que la cerca funciona bé:

1. **Crea un test set:** 10-20 preguntes amb les respostes esperades (quins documents haurien d'aparèixer).
2. **Executa cada pregunta** i mira els resultats.
3. **Calcula mètriques:** - **Recall@K** (quants documents rellevants apareixen entre els K primers resultats). - **MRR** (Mean Reciprocal Rank, rang mitjà del primer document rellevant).
4. **Ajusta:** si els resultats són dolents, canvia de model d'embeddings, ajusta la mida dels fragments, o afegeix metadades.

Per a Hort Osona, les primeres 10 preguntes bones serien:

1. "Quan plantar tomàquets a Osona?"
2. "Com combatre el pugó?"
3. "Quines associacions de cultius funcionen?"
4. "Com fer compost casolà?"

5. "Quan collir les carbasses?"
6. "Quin reg necessita l'enciam?"
7. "Com semblar pèsols?"
8. "Quines plagues ataquen la patata?"
9. "Com protegir les plantes de la gelada?"
10. "Quan fer la poda dels fruiters?"

Si el sistema retorna les fitxes correctes per a totes 10, tens una bona base.

36.11 Com actualitzar la base vectorial

Quan afegixis nous documents a Hort Osona (una nova fitxa, una guia actualitzada), cal re-indexar. Opcions:

1. **Re-indexar tot sencer.** Lent però segur.
2. **Indexar només els nous.** Ràpid, però cal gestionar els fragments antics.
3. **Sistema d'actualització automàtic.** Un script que mira els canvis amb `git pull` i re-indexa el que cal.

Recomanació per al BernatLab: un script setmanal via cron que comprova si hi ha canvis i re-indexa només el que cal.

36.12 Emmagatzematge i persistència

La base vectorial s'ha de desar a algun lloc:

- **Local al Mac:** `~/chromadb` o `./vectorstore`. Fàcil però cal còpia de seguretat.
- **A la Raspberry:** si vols compartir amb altres dispositius.
- **A GitHub:** NO! Els fitxers de la base vectorial ocupen molt i canvien sovint.

Recomanació: **local al Mac, amb còpia a la Raspberry via Tailscale si vols accedir-hi des d'allà.** No versionis la base vectorial a Git.

36.13 Com moure la base vectorial a la Raspberry

Si vols accedir a la base vectorial des de la Raspberry, pots:

1. **Muntar la carpeta** via NFS o SMB (compartir carpetes per xarxa).
2. **Usar SSHFS** (muntar carpetes remotes com si fossin locals).
3. **Exportar/importar** periòdicament (un script que fa `cp`).

L'opció més neta per a un homelab és SSHFS:

```
# A la Raspberry  
sshfs bernat@mac-tailscale-ip>:/Users/bernat/vectorstore /home/bernat/vectorstore
```

Ara `/home/bernat/vectorstore` és la mateixa carpeta que al Mac.

36.14 Resum

Hem après què són els embeddings, com representen el significat del text com a vectors numèrics, com funcionen les bases vectorials, i com indexar els documents d'Hort Osona amb ChromaDB. Al proper capítol muntarem el sistema RAG complet: quan l'usuari fa una pregunta, buscarem els fragments més rellevants i els donarem al model de llengua perquè generi una resposta basada en ells.

36.15 Exercicis pràctics

1. Instal·la ChromaDB al teu Mac: `pip install chromadb`.
2. Descarrega `nomic-embed-text` a Ollama.
3. Executa l'script d'exemple amb 3-5 documents i comprova que la cerca funciona.
4. Adapta l'script `index_hort_osona.py` al teu cas.
5. Crea 10 preguntes de test i avalua la qualitat de la cerca.
6. Compara `nomic-embed-text` amb `bge-m3` (si tens temps).
7. Documenta al README la mida de la base vectorial, el model d'embeddings, i les mètriques obtingudes.

Paraules clau: **embedding, vector, representació vectorial, incrustació, similaritat, cosinus, cosine similarity, distància euclidiana, dot product, base vectorial, vector store, vector database, ChromaDB, FAISS, LanceDB, Qdrant, Milvus, Pinecone, fragmentació, chunking, solapament, overlap, n_results, recall, MRR, mètriques, test set, ground truth, embeddings multilingüe, nomic, bge, e5, mxbai, arctic, multilingual, semàntica, cerca semàntica, cerca per paraules clau, search, full-text search, FTS, BM25, híbrida, reranking, cross-encoder, bi-encoder, contrastive learning, sentence-transformers, Hugging Face, Ollama embedding, API embedding, persistència, SSHFS, NFS, còpia de seguretat, vectorstore, col·lecció, document, fragment, metadata, filtratge, filter, where, where_document, similarity threshold, top-k.**

Capitol 37 — RAG pas a pas: carregar les 76 fitxes d'hort a Ollama

"El RAG és la diferència entre una eina que respon coses generals i una eina que respon coses sobre el teu hort."

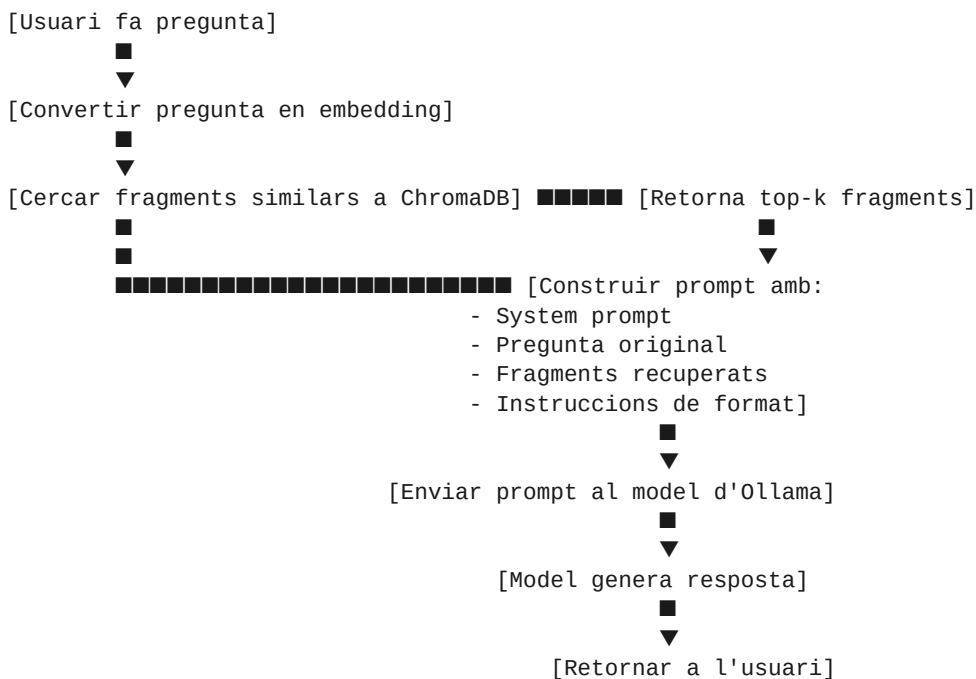
37.1 Què és RAG (revisited)

Al Cap 36 vàrem veure els embeddings i les bases vectorials. Ara les connectem amb un model de llengua per crear un **RAG** (Retrieval-Augmented Generation, generació augmentada per recuperació): quan l'usuari fa una pregunta, primer busquem els fragments més rellevants als documents, i després donem aquests fragments al model com a context perquè generi una resposta.

Això és molt potent perquè:

- El model **no necessita saber** les respostes de memòria.
- Les respostes estan **basades en els teus documents**, no en dades generals.
- Pots **actualitzar els documents** sense re-entrenar el model.
- Les respostes poden **citar** els documents d'origen.

37.2 L'arquitectura d'un sistema RAG



37.3 El primer RAG: versió mínima

Aquí tens un RAG complet en 30 línies de Python:

```
#!/usr/bin/env python3
"""
```

```

rag_simple.py – Primer RAG per a Hort Osona.
Necessita: ollama (servidor corrent), chromadb, requests.
"""

import requests
import chromadb
from chromadb.utils import embedding_functions

# Configuració
MODEL_LLM = "gemma3:12b"
MODEL_EMBEDDING = "nomic-embed-text"
COLLECTION_NAME = "hort_osona"
N_FRAGMENTS = 4 # quants fragments recuperar

# Connexió a ChromaDB
ollama_ef = embedding_functions.OllamaEmbeddingFunction(
    model_name=MODEL_EMBEDDING,
    url="http://localhost:11434/api/embeddings"
)
client = chromadb.PersistentClient(path="./vectorstore")
collection = client.get_or_create_collection(
    name=COLLECTION_NAME,
    embedding_function=ollama_ef
)

def cerca(consulta: str, k: int = N_FRAGMENTS) -> list[str]:
    """Retorna els k fragments més rellevants per a la consulta."""
    resultats = collection.query(query_texts=[consulta], n_results=k)
    return resultats['documents'][0]

def genera_resposta(pregunta: str) -> str:
    """Genera una resposta basada en els fragments recuperats."""
    fragments = cerca(pregunta)
    context = "\n\n".join(
        f"[Fragment {i+1}]: {frag}" for i, frag in enumerate(fragments)
    )
    prompt = f"""Ets l'assistent Hort Osona. Respon en català, basant-te
    només en la informació dels fragments següents. Si la informació no
    és als fragments, digues "No tinc prou informació a les fitxes d'Hort
    Osona per respondre aquesta pregunta."

    FRAGMENTS D'HORT OSONA:
    {context}

    PREGUNTA: {pregunta}

    RESPOSTA: """

    # Crida a l'API d'Ollama
    r = requests.post("http://localhost:11434/api/generate", json={
        "model": MODEL_LLM,
        "prompt": prompt,
        "stream": False
    })
    return r.json()["response"]

if __name__ == "__main__":
    import sys
    pregunta = " ".join(sys.argv[1:]) or "Quan he de plantar carbasses?"
    print(f"\nPregunta: {pregunta}\n")
    print("Resposta:")

```

```
print(genera_resposta(pregunta))
```

Prova'l:

```
python rag_simple.py "Com combatre el pugó?"
python rag_simple.py "Quan plantar tomàquets?"
```

37.4 Millorant el prompt

El prompt de l'apartat anterior és molt bàsic. Un millor prompt:

```
PROMPT_TEMPLATE = """Ets l'assistent del projecte Hort Osona, un hort ecològic a la comarca d'Osona (Catalunya). Tens accés a 76 fitxes de cultius i 30+ guies d'horticultura.
```

```
INSTRUCCIONS:
```

1. Respon SEMPRE en català.
2. Basat-te NOMÉS en la informació dels fragments. Si no hi ha prou informació, digues "No ho sé amb les dades que tinc".
3. Sigues pràctic i directe. Evita frases com "En general..." o "És important...". Dona consells concrets.
4. Si la pregunta és sobre una planta concreta, esmenta la fitxa d'origen.
5. Si la pregunta és general, dona una resposta sintètica (5-10 línies).
6. Si hi ha diverses opcions, enumera-les amb números.

```
FRAGMENTS D'HORT OSONA:
```

```
{context}
```

```
PREGUNTA: {pregunta}
```

```
RESPOSTA: ""
```

Aquest prompt força respostes útils i honestes.

37.5 Afegir cites als documents

Una millora important: retornar **quina fitxa ha donat la informació**. Això permet a l'usuari verificar i aprendre'n més.

```
def cerca_amb_metadades(consulta: str, k: int = 4) -> list[dict]:
    resultats = collection.query(
        query_texts=[consulta],
        n_results=k
    )
    fragments = []
    for i in range(len(resultats['documents'][0])):
        fragments.append({
            "text": resultats['documents'][0][i],
            "font": resultats['metadatas'][0][i]['source'],
            "tema": resultats['metadatas'][0][i].get('tema', 'desconegut'),
            "distancia": resultats['distances'][0][i] if 'distances' in resultats else None
        })
    return fragments

def genera_resposta_amb_cites(pregunta: str) -> dict:
    fragments = cerca_amb_metadades(pregunta)
    context = "\n\n".join(
        f"[{f['tema']} | {f['font']}]: {f['text']}" for f in fragments
    )
    prompt = PROMPT_TEMPLATE.format(context=context, pregunta=pregunta)
```

```

r = requests.post("http://localhost:11434/api/generate", json={
    "model": MODEL_LLM,
    "prompt": prompt,
    "stream": False
})
return {
    "resposta": r.json()["response"],
    "fonts": list(set(f['font'] for f in fragments)),
    "fragments": fragments
}

```

Ara la resposta inclou quines fitxes ha usat.

37.6 Com gestionar fragments massa llargs

Quan recuperes 4-5 fragments, el context pot ser massa llarg per a un model petit. Solucions:

1. **Limitar la mida dels fragments** a l'hora d'indexar (ja ho fem al Cap 36).
2. **Resumir els fragments** amb el model abans de generar la resposta.
3. **Comprimir el context** tècniques com LongLLMLingua, Selective Context, LLMLingua.

Per al BernatLab, la solució 1 ja és bona. Si volem anar més enllà, la 2:

```

def resum_fragments(fragments: list[str], pregunta: str) -> str:
    """Resum cada fragment en una frase, mantenint el relevant per a la pregunta."""
    resums = []
    for frag in fragments:
        prompt = f"Resum en una frase el següent text, mantenint la informació rellevant per a: '{pregunta}'"
        r = requests.post("http://localhost:11434/api/generate", json={
            "model": MODEL_LLM,
            "prompt": prompt,
            "stream": False
        })
        resums.append(r.json()["response"].strip())
    return "\n".join(f"- {r}" for r in resums)

```

Això fa una passada extra pel model, però redueix molt la mida del context final.

37.7 Com afegir metadades útils

Per a Hort Osona, les metadades importants són:

- **Temporada:** primavera, estiu, tardor, hivern.
- **Secció de l'hort:** tomateres, carabasseres, fruiters, compost, etc.
- **Tipus:** fitxa, guia, pla mensual, recepta.
- **Data:** quan s'ha escrit/actualitzat.

Si volem cerques més intel·ligents ("quines coses he de fer aquest mes?"), podem filtrar per temporada:

```

def cerca_filtrada(consulta: str, temporada: str = None, k: int = 4) -> list:
    where = {"temporada": temporada} if temporada else None
    resultats = collection.query(
        query_texts=[consulta],
        n_results=k,
        where=where
    )

```

```
return resultats['documents'][0]
```

Això permet respostes contextualitzades ("a l'estiu fes X, a l'hivern fes Y").

37.8 Com gestionar actualitzacions

Quan escrius una nova fitxa o actualitzes una de vella, cal:

1. **Re-indexar el document sencer** (esborrar els fragments antics i afegir els nous).
2. **Indexar només els canvis** (més eficient, però cal gestionar versions).

Un script de re-indexació per canvi:

```
def reindexar_document(path: Path):
    """Re-indexa un sol document."""
    # Esborrar fragments antics
    tema = path.stem
    try:
        collection.delete(where={"tema": tema})
    except:
        pass

    # Llegir i fragmentar
    text = path.read_text(encoding="utf-8")
    fragments = fragmentar(text)

    # Afegir nous fragments
    for n, frag in enumerate(fragments):
        collection.add(
            documents=[frag],
            metadatas=[{
                "source": str(path.name),
                "fragment": n,
                "tema": tema
            }],
            ids=[f"{tema}-{n}"]
        )
```

I un script setmanal que comprova quins fitxers han canviat:

```
import os
import time
from pathlib import Path

HORT_DIR = Path("/home/bernat/bernatlab/projects/hort-osona")
INDEX_FILE = Path(".indexed_files.txt")

# Carregar l'estat anterior
indexats = {}
if INDEX_FILE.exists():
    for line in INDEX_FILE.read_text().splitlines():
        if "," in line:
            f, mtime = line.split(",")
            indexats[f] = float(mtime)

# Comprovar canvis
nous_canvis = []
for md in HORT_DIR.rglob("*.md"):
    rel = str(md.relative_to(HORT_DIR))
    mtime = md.stat().st_mtime
    if rel not in indexats or mtime > indexats[rel]:
        nous_canvis.append(md)
```



```

        indexats[rel] = mtime

# Re-indexar
for md in nous_canvis:
    print(f"Re-indexant: {md.name}")
    reindexar_document(md)

# Guardar estat
INDEX_FILE.write_text("\n".join(f"{f},{m}" for f, m in indexats.items()))

```

37.9 Com avaluar la qualitat del RAG

Per saber si el teu RAG funciona bé, crea un **test set**:

```

tests = [
    {
        "pregunta": "Quan plantar carbasses?",
        "resposta_esperada": "A la primavera, després de les últimes gelades",
        "fonts_esperades": ["fitxa-carbassa.md", "calendari-sembrar.md"]
    },
    {
        "pregunta": "Com combatre el pugó?",
        "resposta_esperada": "Sabó potàssic, infusions d'all, afavorir marietes",
        "fonts_esperades": ["guia-plagues.md", "gestio-plagues.md"]
    },
    # ...
]

for t in tests:
    r = genera_resposta_amb_cites(t["pregunta"])
    correcte = t["fonts_esperades"][0] in r["fonts"]
    print(f"{'✓' if correcte else '✗'} {t['pregunta']}")

```

Si el RAG falla, ajusta:

- El model d'embeddings (prova'n un altre).
- La mida dels fragments.
- El nombre de fragments recuperats.
- El prompt del sistema.
- La qualitat dels documents d'origen.

37.10 Com passar a streaming

Per a respostes llargues, voldrem **streaming** (rebre la resposta a mesura que es genera, en comptes d'esperar tota la resposta):

```

import json

def genera_stream(pregunta: str):
    fragments = cerca_amb_metadades(pregunta)
    context = "\n\n".join(f"[{f['font']}]: {f['text']}" for f in fragments)
    prompt = PROMPT_TEMPLATE.format(context=context, pregunta=pregunta)

    with requests.post(
        "http://localhost:11434/api/generate",
        json={"model": MODEL_LLM, "prompt": prompt, "stream": True},
        stream=True
    ) as response:
        for chunk in response.iter_lines():
            if chunk:
                data = json.loads(chunk.decode('utf-8'))
                yield data["text"]

```

```
) as r:
    for line in r.iter_lines():
        if line:
            data = json.loads(line)
            if "response" in data:
                yield data["response"]
```

Ús:

```
for chunk in genera_stream("Quan plantar carbasses?"):
    print(chunk, end="", flush=True)
print()
```

Això és el que farem servir al client web (Cap 38).

37.11 Errors habituals

Error 1: el model no segueix les instruccions del prompt.

Solució: fes servir un model més gran o reescriu el prompt més clar.

Error 2: la cerca retorna fragments irrelevants.

Solució: canvia el model d'embeddings, millora els documents, redueix k.

Error 3: les respostes són molt llargues.

Solució: afegeix "Respon en 3-5 línies" al prompt.

Error 4: les respostes inventen informació.

Solució: reforça el prompt amb "Si no saps, digues 'no ho sé'". Afegeix cites per verificar.

Error 5: la base vectorial creix molt.

Solució: fragmenta més fi (300-400 tokens) o redueix el nombre de documents.

37.12 Resum

Hem après a construir un RAG complet amb Ollama i ChromaDB: com recuperar fragments, com construir el prompt, com generar respostes amb cites, com gestionar actualitzacions, i com avaluar la qualitat. Al proper capítol construirem un client web senzill perquè puguis parlar amb l'assistent des del navegador.

37.13 Exercicis pràctics

1. Descarrega el codi de `rag_simple.py` i prova'l amb 5 preguntes.
2. Avalua la qualitat de les respostes.
3. Afegeix cites als documents.
4. Implementa streaming.
5. Crea un test set amb 10 preguntes i avalua el rendiment.
6. Ajusta el prompt per millorar les respostes.
7. Documenta al README la configuració del RAG, el model usat, i les mètriques.

Paraules clau: **RAG, retrieval-augmented generation, generació augmentada, fragments, top-k, context, prompt, system prompt, streaming, cites, fonts, metadades, ChromaDB, Ollama, embedding, on-premises, self-hosted, private AI, hort Osona, fitxes, guies, re-indexació, actualització, test set, avaluació, mètriques, recall, precision, MRR, reranking, re-ranking, rerank, cross-encoder, bi-encoder, hybrid search, BM25, filtratge, where, chunking, fragmentació, size, overlap, sliding window, semantic chunking, structured chunking, LongLLMLingua, LLMLingua, context compression, summarization, query expansion, HyDE, hypothetical document embeddings, multi-query, fusion, RAG-Fusion, agentic RAG, ReAct, self-RAG, corrective RAG, CRAG, advanced RAG, modular RAG, GraphRAG, knowledge graph.**

Capitol 38 — Client web

""Un assistent potent al terminal és útil. Un assistent al navegador, amb historial, cites i veu, és una altra cosa.""

38.1 Per què un client web

Un cop tens el RAG funcionant al terminal (Cap 37), vols alguna cosa més usable:

- **Interfície gràfica** amb quadre de text, historial, format.
- **Cites clicables** que porten a la fitxa original.
- **Markdown renderitzat** (l·listes, negretes, enllaços).
- **Codi amb color** (útil si preguntes sobre scripts).
- **Persistència** de l'historial entre sessions.
- **Accessible des de qualsevol dispositiu** del tailnet.

La manera més senzilla és un client web local que connecta amb Ollama. Sense dependències complexes, en HTML + JavaScript, allotjable a qualsevol lloc.

38.2 Stack tecnològic

Triarem eines lleugeres, sense frameworks pesats:

- **Backend:** FastAPI (Python) — simple, modern, async.
- **Frontend:** HTML + CSS + JavaScript — sense React ni Vue, perquè volem poc.
- **Markdown:** marked.js (llibreria que converteix Markdown a HTML, ~30 KB) carregada per CDN.
- **Syntax highlight:** highlight.js per al codi, per CDN.

Tot plegat: **150 KB de frontend, 200 línies de Python al backend**. Prou per a un assistent potent.

38.3 Estructura del projecte

```
book/asistent/
├── backend/
│   ├── main.py           # FastAPI app
│   ├── rag.py            # Lògica RAG (Cap 37)
│   └── requirements.txt  # Dependències
├── frontend/
│   ├── index.html        # Pàgina principal
│   ├── styles.css        # Estils
│   └── app.js            # Lògica del client
└── README.md            # Com arrancar-ho
```

38.4 Backend: FastAPI

Crea `backend/main.py`:

```
#!/usr/bin/env python3
"""
```

asistent_hort_osona.py – Backend FastAPI per a l'assistent Hort Osona.

```
"""

from fastapi import FastAPI, HTTPException
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from pydantic import BaseModel
from pathlib import Path
import sys

# Afegir el directori del RAG al path
sys.path.insert(0, str(Path(__file__).parent))
from rag import HortOsonaRAG

# Inicialitzar
app = FastAPI(title="Hort Osona Assistant")
rag = HortOsonaRAG(
    model_llm="gemma3:12b",
    model_embedding="nomic-embed-text",
    collection_name="hort_osona",
    vectorstore_path="./vectorstore"
)

# Servir fitxers estàtics
FRONTEND_DIR = Path(__file__).parent.parent / "frontend"
app.mount("/static", StaticFiles(directory=FRONTEND_DIR), name="static")

class Pregunta(BaseModel):
    text: str
    stream: bool = True

@app.get("/")
def root():
    return FileResponse(FRONTEND_DIR / "index.html")

@app.get("/api/preguntar")
def preguntar_text(q: str):
    """Endpoint per streaming amb Server-Sent Events (SSE)."""
    from fastapi.responses import StreamingResponse
    import json

    def generator():
        for chunk in rag.ask_stream(q):
            yield f"data: {json.dumps(chunk)}\n\n"
        yield "data: [DONE]\n\n"

    return StreamingResponse(generator(), media_type="text/event-stream")

@app.post("/api/preguntar")
def preguntar_json(pregunta: Pregunta):
    """Endpoint per obtenir resposta completa en JSON."""
    if pregunta.stream:
        raise HTTPException(400, "Ureu /api/preguntar?q=... per streaming")
    return rag.ask(pregunta.text)

@app.get("/api/estadistiques")
def estadistiques():
    """Retorna estadístiques de la base vectorial."""
    return {
```

```

    "fragments_indexats": rag.count(),
    "model_llm": rag.model_llm,
    "model_embedding": rag.model_embedding,
}

```

38.5 Backend: la classe RAG

Crea `backend/rag.py` (basat en el Cap 37, però organitzat com a classe):

```

"""
rag.py – Lògica del RAG per a Hort Osona.
"""

import json
from pathlib import Path
import chromadb
from chromadb.utils import embedding_functions
import requests

class HortOsonaRAG:
    def __init__(self, model_llm, model_embedding, collection_name, vectorstore_path):
        self.model_llm = model_llm
        self.model_embedding = model_embedding

        # Configurar embeddings via Ollama
        ollama_ef = embedding_functions.OllamaEmbeddingFunction(
            model_name=model_embedding,
            url="http://localhost:11434/api/embeddings"
        )

        # Connectar a ChromaDB
        client = chromadb.PersistentClient(path=vectorstore_path)
        self.collection = client.get_or_create_collection(
            name=collection_name,
            embedding_function=ollama_ef
        )

    def cerca(self, pregunta: str, k: int = 4) -> list[dict]:
        resultats = self.collection.query(
            query_texts=[pregunta],
            n_results=k
        )
        return [
            {
                "text": resultats['documents'][0][i],
                "font": resultats['metadatas'][0][i].get('source', 'desconegut'),
                "tema": resultats['metadatas'][0][i].get('tema', ''),
                "distancia": resultats['distances'][0][i] if 'distances' in resultats else 0
            }
            for i in range(len(resultats['documents'][0]))
        ]

    def _build_prompt(self, pregunta: str, fragments: list[dict]) -> str:
        context = "\n\n".join(
            f"[Fragment de {f['font']}] \n {f['text']}"
            for f in fragments
        )
        return f"""Ets l'assistent del projecte Hort Osona.

```

INSTRUCCIONS:

- Respon SEMPRE en català.
- Basat-te NOMÉS en la informació dels fragments.

- Si la informació no és als fragments, digues "No ho sé amb les dades d'Hort Osona".
- Sigues pràctic i concret.
- Si la pregunta és sobre una planta concreta, esmenta la fitxa d'origen.

FRAGMENTS:

{context}

PREGUNTA: {pregunta}

RESPOSTA: ""

```
def ask(self, pregunta: str) -> dict:
    fragments = self.cerca(pregunta)
    prompt = self._build_prompt(pregunta, fragments)
    r = requests.post("http://localhost:11434/api/generate", json={
        "model": self.model_llm,
        "prompt": prompt,
        "stream": False
    })
    return {
        "resposta": r.json()["response"],
        "fonts": list(set(f['font'] for f in fragments)),
        "fragments": fragments
    }

def ask_stream(self, pregunta: str):
    """Genera la resposta en streaming, amb cites al final."""
    fragments = self.cerca(pregunta)
    prompt = self._build_prompt(pregunta, fragments)

    # Enviar primer les cites
    yield {
        "type": "fonts",
        "fonts": list(set(f['font'] for f in fragments))
    }

    # Després la resposta en streaming
    with requests.post(
        "http://localhost:11434/api/generate",
        json={"model": self.model_llm, "prompt": prompt, "stream": True},
        stream=True
    ) as r:
        for line in r.iter_lines():
            if line:
                data = json.loads(line)
                if "response" in data:
                    yield {
                        "type": "text",
                        "content": data["response"]
                    }

def count(self) -> int:
    return self.collection.count()
```

38.6 Frontend: HTML

Crea frontend/index.html:

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <title>Hort Osona · Assistent</title>
```

```

<link rel="stylesheet" href="/static/styles.css">
<script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/dompurify@3.0.6/dist/purify.min.js"></script>
<link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/highlight.js@11.9.0/styles/github.min.css">
<script src="https://cdn.jsdelivr.net/npm/highlight.js@11.9.0/lib/core.min.js"></script>
</head>
<body>
  <header>
    <h1>■ Hort Osona</h1>
    <p class="subtitle">Assistent local basat en les teves 76 fitxes</p>
  </header>

  <main>
    <div id="chat"></div>

    <form id="form">
      <textarea
        id="input"
        placeholder="Pregunta'm sobre el teu hort..."
        rows="3"
        autofocus></textarea>
      <div class="actions">
        <span id="status"></span>
        <button type="submit">Enviar</button>
      </div>
    </form>
  </main>

  <footer>
    <small>
      Model: <span id="model-name">...</span> .
      Fragments: <span id="fragments-count">...</span>
    </small>
  </footer>

  <script src="/static/app.js"></script>
</body>
</html>

```

38.7 Frontend: CSS

Crea frontend/styles.css:

```

* { box-sizing: border-box; }
body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  margin: 0;
  background: #f8f9fa;
  color: #222;
  line-height: 1.6;
}
header {
  background: #2d5016;
  color: #fff;
  padding: 1em 2em;
  text-align: center;
}
header h1 { margin: 0; font-size: 1.5em; }
header .subtitle { margin: 0.3em 0 0; opacity: 0.8; font-size: 0.9em; }
main {
  max-width: 800px;
  margin: 0 auto;
}

```



```
padding: 1em;
min-height: 70vh;
display: flex;
flex-direction: column;
}
#chat {
  flex: 1;
  overflow-y: auto;
  margin-bottom: 1em;
}
.message {
  margin: 1em 0;
  padding: 0.8em 1em;
  border-radius: 8px;
  max-width: 90%;
}
.user {
  background: #e3f2fd;
  margin-left: auto;
}
.assistant {
  background: #fff;
  border: 1px solid #e0e0e0;
}
.assistant pre {
  background: #1e1e1e;
  color: #d4d4d4;
  padding: 0.8em;
  border-radius: 4px;
  overflow-x: auto;
}
.assistant code {
  background: #f0f0f0;
  padding: 0.1em 0.3em;
  border-radius: 3px;
}
.fonts {
  margin-top: 0.5em;
  font-size: 0.85em;
  color: #666;
  font-style: italic;
}
#form {
  background: #fff;
  padding: 1em;
  border-radius: 8px;
  border: 1px solid #e0e0e0;
}
#input {
  width: 100%;
  border: 1px solid #ddd;
  border-radius: 4px;
  padding: 0.5em;
  font-family: inherit;
  font-size: 1em;
  resize: vertical;
}
.actions {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 0.5em;
}
button {
```

```

    background: #2d5016;
    color: #fff;
    border: none;
    padding: 0.5em 1.5em;
    border-radius: 4px;
    cursor: pointer;
    font-size: 1em;
}
button:hover { background: #3d6b1f; }
button:disabled { background: #aaa; cursor: not-allowed; }
#status { font-size: 0.85em; color: #666; }
footer {
    text-align: center;
    color: #888;
    padding: 1em;
}

```

38.8 Frontend: JavaScript

Crea frontend/app.js:

```

// app.js – Client per a l'assistent Hort Osona

const chat = document.getElementById('chat');
const form = document.getElementById('form');
const input = document.getElementById('input');
const status = document.getElementById('status');
const modelName = document.getElementById('model-name');
const fragmentsCount = document.getElementById('fragments-count');

// Carregar estadístiques
fetch('/api/estadistiques')
    .then(r => r.json())
    .then(d => {
        modelName.textContent = d.model_llm;
        fragmentsCount.textContent = d.fragments_indexats;
    });

function afegirMissatge(text, role, fonts = null) {
    const div = document.createElement('div');
    div.className = `message ${role}`;
    if (role === 'assistant') {
        div.innerHTML = DOMPurify.sanitize(marked.parse(text));
        if (fonts && fonts.length > 0) {
            const f = document.createElement('div');
            f.className = 'fonts';
            f.textContent = '■ ' + fonts.join(', ');
            div.appendChild(f);
        }
    } else {
        div.textContent = text;
    }
    chat.appendChild(div);
    chat.scrollTop = chat.scrollHeight;
    return div;
}

form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const text = input.value.trim();
    if (!text) return;

    afegirMissatge(text, 'user');

```

```

input.value = '';
status.textContent = 'Pensant...';
form.querySelector('button').disabled = true;

try {
  // Obrir stream
  const response = await fetch(`/api/preguntar?q=${encodeURIComponent(text)}`);
  const reader = response.body.getReader();
  const decoder = new TextDecoder();

  let buffer = '';
  let assistantDiv = null;
  let fullText = '';
  let fonts = null;

  while (true) {
    const { value, done } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });

    // Processar línies SSE
    const lines = buffer.split('\n\n');
    buffer = lines.pop() || '';

    for (const line of lines) {
      if (!line.startsWith('data: ')) continue;
      const data = line.slice(6);
      if (data === '[DONE]') continue;

      try {
        const obj = JSON.parse(data);
        if (obj.type === 'fonts') {
          fonts = obj.fonts;
        } else if (obj.type === 'text') {
          fullText += obj.content;
          if (!assistantDiv) {
            assistantDiv = afegirMissatge('', 'assistant');
          }
          assistantDiv.innerHTML = DOMPurify.sanitize(
            marked.parse(fullText)
          );
        }
      } catch (e) {}
    }
    chat.scrollTop = chat.scrollHeight;
  }

  // Afegir cites al final
  if (fonts && assistantDiv) {
    const f = document.createElement('div');
    f.className = 'fonts';
    f.textContent = '■ Fonts: ' + fonts.join(', ');
    assistantDiv.appendChild(f);
  }

  status.textContent = '';
} catch (err) {
  afegirMissatge('Error: ' + err.message, 'assistant');
  status.textContent = '';
} finally {
  form.querySelector('button').disabled = false;
}
});

```

38.9 Com arrencar-ho tot

1. Assegura't que Ollama està corrent:

```
ollama serve &
```

1. Indexa les fitxes (només la primera vegada, o quan n'afegim de noves):

```
python backend/./index_hort_osona.py
```

1. Arrenca el backend:

```
cd backend
pip install -r requirements.txt # fastapi, uvicorn, chromadb, requests
uvicorn main:app --host 0.0.0.0 --port 8080 --reload
```

1. Obre el navegador a <http://localhost:8080>.

38.10 Accedir des de la Raspberry o el mòbil

Si vols accedir des d'altres dispositius del tailnet:

1. Assegura't que el backend escolta a 0.0.0.0 (ja ho fem amb `--host 0.0.0.0`).

2. Obre el port al firewall del Mac:

```
# System Settings → Network → Firewall → Options
# Afegir uvicorn (Python) i permetre connexions entrants
```

1. Accedeix des de la Raspberry:

```
curl http://<mac-tailscale-ip>:8080/api/estadistiques
```

1. Des del navegador del mòbil: obre <http://<mac-tailscale-ip>:8080>.

38.11 Millores opcionals

Un cop el bàsic funciona, pots afegir:

1. **Historial persistent.** Guarda les converses a una base de dades SQLite o fitxer JSON.
2. **Sistema de feedback.** Botons   per millorar el model.
3. **Memòria a llarg termini.** Recorda les preferències de l'usuari (l'hort, varietats favorites).
4. **Exportar converses.** Descarrega les respostes com a Markdown o PDF.
5. **Multimodal.** Afegeix lectura d'imatges (p. ex. una foto d'una planta malalta).
6. **Veu.** Cap 40.

38.12 Desplegament al BernatLab

Si vols que el client estigui disponible sempre, sense haver d'arrencar-lo manualment:

1. Crea un servei launchd al Mac:

```
<!-- ~/Library/LaunchAgents/com.bernat.asistent.plist -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>com.bernat.asistent</string>
  <key>ProgramArguments</key>
  <array>
    <string>/Users/bernat/bernatlab/.venv/bin/python</string>
    <string>-m</string>
    <string>uvicorn</string>
    <string>main:app</string>
    <string>--host</string>
    <string>0.0.0.0</string>
    <string>--port</string>
    <string>8080</string>
  </array>
  <key>WorkingDirectory</key>
  <string>/Users/bernat/bernatlab/asistent/backend</string>
  <key>RunAtLoad</key>
  <true/>
  <key>KeepAlive</key>
  <true/>
</dict>
</plist>
```

1. Carrega'l:

```
launchctl load ~/Library/LaunchAgents/com.bernat.asistent.plist
```

Ara l'assistent arrenca automàticament quan encens el Mac.

38.13 Resum

Hem muntat un client web complet per a l'assistent Hort Osona: backend FastAPI, frontend HTML+CSS+JS, streaming de respostes, cites als documents, i persistència. Al proper capítol veurem com integrar Ollama amb l'API del BernatLab perquè la Raspberry pugui preguntar coses al Mac i rebre respostes basades en les fitxes.

38.14 Exercicis pràctics

1. Crea l'estructura `asistent/backend/` i `asistent/frontend/`.
2. Escribeu `rag.py` i `main.py`.
3. Escribeu `index.html`, `styles.css`, i `app.js`.
4. Arrenca Ollama i el backend amb `uvicorn`.
5. Obre `http://localhost:8080` i fes 5 preguntes.
6. Afegeix el servei `launchd` per arrencada automàtica.
7. Configura l'accés des de la Raspberry via `Tailscale`.
8. Documenta al `README` com usar-lo.

Paraules clau: **FastAPI, uvicorn, streaming, Server-Sent Events, SSE, EventSource, ReadableStream, frontend, HTML, CSS, JavaScript, marked.js, DOMPurify, highlight.js, marked, sanitize, Markdown, renderitzat, historial, persistència, localStorage, SQLite, JSON, feedback, usuari, multi-dispositiu, Tailscale, mòbil, responsive, launchd, plist, dimoni, servei, arrencada automàtica, RunAtLoad, KeepAlive, Mac, deployment, producció, monitor, watchdog, pàgina web, navegador, client, UI, UX, interfície, quadre de text, Enter, enviar, cites, fonts, links, click, enllaços, navegació.**

Capitol 39 — API d'Ollama: integrar la IA amb el BernatLab

"Ollama al Mac, el BernatLab a la Raspberry. Si parlen entre ells, tens un sistema que pot raonar sobre les dades del teu hort sense sortir de casa."

39.1 L'API d'Ollama

Ollama exposa una **API HTTP local** (a `http://localhost:11434`) que permet:

- Generar respostes (`/api/generate`).
- Mantenir converses amb historial (`/api/chat`).
- Generar embeddings (`/api/embeddings`).
- Llistar models (`/api/tags`).
- Aturar/reprendre models (`/api/ps`, `/api/load`, `/api/unload`).

També és **compatible amb l'API d'OpenAI**, així que qualsevol eina que funcioni amb OpenAI funciona amb Ollama (canviant la URL base).

Això obre la porta a integrar Ollama amb tot el que ja tenim al BernatLab: FastAPI, Node-RED, scripts Python, Telegram bots, etc.

39.2 Configurar Ollama per accedir des de la xarxa

Per defecte, Ollama escolta a `localhost:11434`. Per permetre connexions des d'altres dispositius:

```
# Atura Ollama
pkill ollama

# Re-arrenca escoltant a totes les interfícies
OLLAMA_HOST=0.0.0.0:11434 ollama serve &
```

Per fer-ho persistent, edita la configuració del servei launchd (macOS):

```
<key>EnvironmentVariables</key>
<dict>
  <key>OLLAMA_HOST</key>
  <string>0.0.0.0:11434</string>
</dict>
```

Verifica des d'un altre dispositiu:

```
curl http://<mac-tailscale-ip>:11434/api/tags
```

Hauries de veure la llista de models.

39.3 Endpoints essencials

Llistar models disponibles

```
curl http://localhost:11434/api/tags
```

Resposta:

```
{
  "models": [
    {
      "name": "gemma3:12b",
      "size": 8149502976,
      "modified_at": "2026-07-08T10:30:00Z"
    }
  ]
}
```

Generar una resposta (no streaming)

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3:12b",
  "prompt": "Quan plantar tomàquets?",
  "stream": false
}'
```

Resposta:

```
{
  "model": "gemma3:12b",
  "response": "Els tomàquets es planten a la primavera...",
  "done": true,
  "context": [123, 456, 789, ...],
  "total_duration": 5234567890,
  "load_duration": 1234567890,
  "prompt_eval_count": 12,
  "eval_count": 87
}
```

Generar amb streaming

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3:12b",
  "prompt": "Explica el compostatge",
  "stream": true
}'
```

Resposta: cada línia és un JSON amb un tros de la resposta. Acaba amb "done": true.

Chat amb historial

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3:12b",
  "messages": [
    {"role": "user", "content": "Hola"},
    {"role": "assistant", "content": "Hola! Com puc ajudar-te?"},
    {"role": "user", "content": "Quan plantar carbasses?"}
  ],
  "stream": false
}'
```

Això manté el context de la conversa.

Generar embeddings

```
curl http://localhost:11434/api/embeddings -d '{
  "model": "nomic-embed-text",
  "prompt": "El tomàquet prefereix sòl ben drenat"
}'
```

Resposta: un vector de 768 números.

39.4 Compatibilitat amb OpenAI

L'endpoint `http://localhost:11434/v1/chat/completions` és compatible amb l'API d'OpenAI. Això vol dir que qualsevol eina que usi OpenAI pot usar Ollama simplement canviant la URL base:

```
# Abans amb OpenAI:
import openai
client = openai.OpenAI(api_key="sk-...")
response = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Hola"}]
)

# Ara amb Ollama (canvi mínim):
import openai
client = openai.OpenAI(
    api_key="ollama", # qualsevol valor, no es valida
    base_url="http://localhost:11434/v1"
)
response = client.chat.completions.create(
    model="gemma3:12b",
    messages=[{"role": "user", "content": "Hola"}]
)
```

Això permet usar eines com:

- **LangChain** (framework per construir aplicacions amb LLMs).
- **LlamaIndex** (framework especialitzat en RAG).
- **Open WebUI** (interfície web similar a ChatGPT).
- **AnythingLLM** (una altra interfície).

39.5 Integració amb la Raspberry

A la Raspberry, podem fer consultes a Ollama al Mac via Tailscale. Un script Python:

```
#!/usr/bin/env python3
"""
consulta_hort.py – Script per fer consultes a Ollama des de la Raspberry.
"""

import requests
import sys

OLLAMA_URL = "http://100.115.134.76:11434" # Mac Tailscale IP
MODEL = "gemma3:12b"

def pregunta(text: str) -> str:
    """Envia una pregunta a Ollama i retorna la resposta."""
    r = requests.post(
        f"{OLLAMA_URL}/api/generate",
        json={
            "model": MODEL,
            "prompt": text,
            "stream": False
        },
        timeout=60
    )
```

```

    )
    r.raise_for_status()
    return r.json()["response"]

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Ús: consulta_hort.py 'la teva pregunta'")
        sys.exit(1)
    q = " ".join(sys.argv[1:])
    print(f"Pregunta: {q}\n")
    print(f"Resposta: {pregunta(q)}")

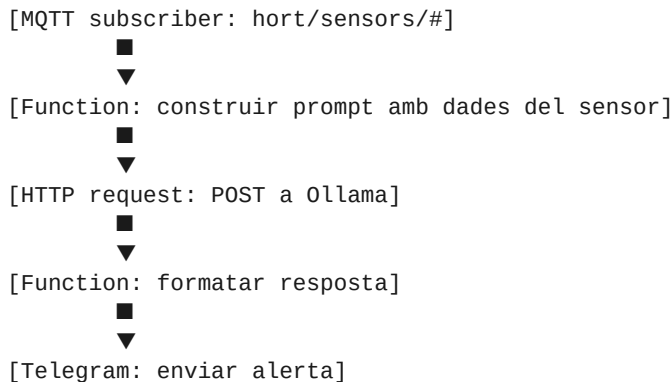
```

Ús:

```
python3 consulta_hort.py "Quan he de plantar carbasses a Osona?"
```

39.6 Integració amb Node-RED

Node-RED té nodes per cridar HTTP. Un flux per rebre alertes de sensors i consultar Ollama:



Exemple del node function que construeix el prompt:

```

// Rep un missatge MQTT amb dades del sensor
const sensorData = msg.payload;

const prompt = `Ets l'assistent de l'hort. Acabo de rebre les següents dades
d'un sensor a l'hort:

${JSON.stringify(sensorData, null, 2)}

Analitza aquestes dades i, si hi ha alguna cosa que requereixi atenció
(temperatura extrema, humitat del sòl massa baixa, bateria baixa),
dona'm una alerta curta en català. Si tot és normal, digue'm "Tot correcte".

Resposta (màxim 3 línies):`;

msg.payload = prompt;
return msg;

```

39.7 Integració amb el bot de Telegram

Tens un bot de Telegram al BernatLab (M2). Pots afegir una comanda `/hort` que consulta Ollama:

```

# Dins del teu bot de Telegram (python-telegram-bot)
async def hort(update, context):
    """Respon a la comanda /hort amb una consulta a Ollama."""
    if not context.args:
        await update.message.reply_text(

```

```

        "Ús: /hort la teva pregunta sobre l'hort"
    )
    return

pregunta = " ".join(context.args)
await context.bot.send_chat_action(
    update.effective_chat.id, "typing"
)

try:
    resposta = pregunta_ollama(pregunta)
    await update.message.reply_text(resposta, parse_mode="Markdown")
except Exception as e:
    await update.message.reply_text(f"Error: {e}")

```

Ara pots fer `/hort Quan plantar tomàquets?` des del mòbil.

39.8 Integració amb FastAPI (BernatLab API)

L'API FastAPI del M2 (Cap 20) pot tenir un nou endpoint que consulta Ollama:

```

# A l'API del BernatLab
from fastapi import APIRouter
import httpx

router = APIRouter()

OLLAMA_URL = "http://100.115.134.76:11434" # Mac Tailscale

@router.get("/hort/preguntar")
async def hort_preguntar(q: str):
    """Consulta l'assistent Hort Osona."""
    async with httpx.AsyncClient(timeout=60) as client:
        r = await client.post(
            f"{OLLAMA_URL}/api/generate",
            json={"model": "gemma3:12b", "prompt": q, "stream": False}
        )
        return r.json()

```

Ara la web pública d'Hort Osona pot tenir un quadre de preguntes que crida aquest endpoint.

39.9 Com gestionar la càrrega

Ollama pot gestionar una sola petició a la vegada bé, però si tens moltes peticions simultànies, cal:

1. **Posar un timeout.** Si la resposta triga més de 60 segons, cancel·la.
2. **Posar una cua.** Usa Redis o RabbitMQ per serialitzar les peticions.
3. **Limitar les peticions concurrents.** Usa un middleware al backend.
4. **Servir el model més petit per a tasques fàcils.** Tenir 2-3 models i triar el adequat.

Exemple de cua simple amb Python:

```

import asyncio
from collections import deque

class OllamaQueue:
    def __init__(self):
        self.cua = deque()
        self.busy = False

```

```

async def processa(self, prompt: str) -> str:
    future = asyncio.Future()
    self.cua.append((prompt, future))
    await self._buidar()
    return await future

async def _buidar(self):
    if self.busy or not self.cua:
        return
    self.busy = True
    while self.cua:
        prompt, future = self.cua.popleft()
        try:
            resultat = await self._trucar_ollama(prompt)
            future.set_result(resultat)
        except Exception as e:
            future.set_exception(e)
    self.busy = False

```

39.10 Monitoratge de l'API

Per saber com va l'API, podem afegir:

1. **Logs:** totes les peticions amb temps de resposta.
2. **Mètriques:** nombre de peticions, latència, errors.
3. **Alertes:** si el model no respon, o si la latència és massa alta.

Exemple amb un middleware de FastAPI:

```

import time
from fastapi import Request

@app.middleware("http")
async def timing(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    duration = time.time() - start
    print(f"{request.method} {request.url.path}: {duration:.2f}s")
    return response

```

I un panell a Uptime Kuma per veure si l'API està viva:

```

# Afegir a Uptime Kuma
curl -X POST http://localhost:3001/api/monitors \
  -H "Content-Type: application/json" \
  -d '{"name": "Ollama API", "url": "http://mac:11434/api/tags", "interval": 60}'

```

39.11 Seguretat de l'API

L'API d'Ollama **no té autenticació** per defecte. Si l'exposes a la xarxa, algú podria:

- Usar el teu Mac per a inferència (consumint CPU/RAM).
- Enviar prompts maliciosos.
- Saturar el sistema.

Per protegir-la:

1. **Tallafocs al Mac:** permet només la xarxa Tailscale.

2. **Reverse proxy amb Nginx/Caddy** i autenticació bàsica.

3. **API key** personalitzada: modifica Ollama per afegir un header d'autenticació.

Una solució senzilla amb Caddy:

```
# /etc/caddy/Caddyfile
ollama.bernat.local {
    basicauth {
        bernat $2a$14$... # hash bcrypt
    }
    reverse_proxy localhost:11434
}
```

Ara cal usuari i contrasenya per accedir.

39.12 Backup i recuperació

Quan tens un sistema que depèn d'Ollama, és bona pràctica:

1. **Guardar els models** amb regularitat: les descàrregues ocupen molt, però si les perds, cal esperar hores a tornar-les a descarregar.
2. **Guardar la base vectorial** (ChromaDB): una còpia de seguretat setmanal.
3. **Guardar l'historial de converses**: si en tens.
4. **Guardar les configuracions**: Modelfiles, scripts, etc.

Un script de backup:

```
#!/bin/bash
# backup_ollama.sh
BACKUP_DIR="/Users/bernat/backups/ollama"
DATE=$(date +%Y%m%d)

mkdir -p "$BACKUP_DIR/$DATE"

# Models
cp -r ~/.ollama "$BACKUP_DIR/$DATE/"

# Base vectorial
cp -r ~/bernatlab/asistent/vectorstore "$BACKUP_DIR/$DATE/"

# Configuració
cp ~/Library/LaunchAgents/com.bernat.asistent.plist "$BACKUP_DIR/$DATE/"

echo "Backup fet a $BACKUP_DIR/$DATE"
```

Executar amb cron o launchd.

39.13 Resum

Hem après a fer accessible l'API d'Ollama des de la xarxa, a integrar-la amb la Raspberry, Node-RED, el bot de Telegram, i l'API del BernatLab. Hem vist com gestionar la càrrega, monitorar, securitzar, i fer còpies de seguretat. Al proper capítol afegirem veu: li parlarem a l'assistent en lloc d'escriure.

39.14 Exercicis pràctics

1. Configura Ollama per escoltar a `0.0.0.0:11434`.
2. Verifica que la Raspberry pot accedir-hi via Tailscale.
3. Crea el script `consulta_hort.py` i prova'l.
4. Afegeix un node HTTP a Node-RED que consulti Ollama.
5. Afegeix una comanda `/hort` al bot de Telegram.
6. Afegeix un endpoint `/hort/preguntar` a l'API FastAPI.
7. Configura Uptime Kuma per monitorar l'API.
8. Fes un backup programat d'Ollama i la base vectorial.

Paraules clau: **API, HTTP, REST, Ollama, localhost, 11434, Tailscale, 100.115.134.76, OpenAI compatibility, /v1, /api/generate, /api/chat, /api/embeddings, /api/tags, streaming, JSON, pydantic, FastAPI, uvicorn, async, httpx, timeout, cua, queue, rate limit, autenticació, basicauth, Caddy, Nginx, seguretat, tallafofs, firewall, monitoratge, Uptime Kuma, alerting, backup, recuperació, restore, snapshot, launchd, servei, dimoni, plist, RunAtLoad, KeepAlive, integració, Node-RED, Telegram, Python, script, CLI, command-line, asyncio, future, deque, prometheus, grafana, observabilitat, mètriques, logs, latència, throughput, peticions per segon, RPS, error rate, time series, panell, alert, contact point.**

Capitol 40 — Veu: parlar a l'assistent

*""Quan tens les mans plenes de terra i el cap ple de preguntes, l'últim que vols és teclejar. Li parles, et contesta. Màgia."**

40.1 Per què veu a Hort Osona

A l'hort, les mans estan ocupades (pala, planter, regadora). El mòbil pot estar brut. Però la veu sempre és disponible. Per això, la integració de veu és especialment valuosa per a consultes ràpides mentre treballes.

Casos d'ús reals:

- Estàs trasplantant tomàquets i vols saber la distància exacta. Li preguntes, et contesta.
- Vius una plaga sobtada i vols consell immediat. Li parles, reps la resposta.
- Estàs collint i vols saber si ja estan al punt. Li ensenyes la foto (multimodal), et diu.

40.2 Stack tecnològic

Per a la veu, usarem:

- **Whisper** (OpenAI) per a **STT** (Speech-to-Text, transcripció de veu a text). Versió local, multilingüe, excel·lent en català.
- **Piper** o **XTTS** per a **TTS** (Text-to-Speech, síntesi de veu). Sintetitza veu a partir de text. Opcions locals, en diversos idiomes.
- **FastAPI** com a backend (reutilitzem el del Cap 38).
- **Web Audio API** al frontend per gravar àudio del micròfon.

Tot local, sense enviar res al núvol.

40.3 Instal·lar Whisper

Whisper és la millor opció per a transcripció. Versió local amb Python:

```
# Opció 1: faster-whisper (recomanada, més ràpida)
pip install faster-whisper
```

```
# Opció 2: openai-whisper (oficial)
pip install openai-whisper
```

faster-whisper és ~4x més ràpida i consumeix menys memòria. Recomanada.

Descarrega un model:

```
from faster_whisper import WhisperModel

# Models disponibles: tiny, base, small, medium, large-v3
# - tiny: 39M paràmetres, ~75 MB, molt ràpid, baixa qualitat
# - base: 74M, ~140 MB, ràpid, qualitat acceptable
# - small: 244M, ~460 MB, equilibrat
# - medium: 769M, ~1.5 GB, bona qualitat
```

```
# - large-v3: 1.5 GB, ~3 GB, millor qualitat, multilingüe

model = WhisperModel("large-v3", device="cpu", compute_type="int8")
```

40.4 Ús bàsic de Whisper

```
from faster_whisper import WhisperModel

model = WhisperModel("large-v3", device="cpu", compute_type="int8")

# Transcriure un fitxer d'àudio
segments, info = model.transcribe("audio.wav", language="ca")

print(f"Idioma detectat: {info.language} (probabilitat: {info.language_probability:.2f})")
print(f"Durada: {info.duration:.1f}s")
print()

for segment in segments:
    print(f"[{segment.start:.1f}s - {segment.end:.1f}s] {segment.text}")
```

Whisper detecta automàticament l'idioma, però pots forçar-lo amb `language="ca"`.

40.5 Capturar àudio del navegador

Al frontend, podem capturar àudio del micròfon amb la **Web Audio API + MediaRecorder**:

```
async function gravarAudio() {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    const recorder = new MediaRecorder(stream);
    const chunks = [];

    recorder.ondataavailable = e => chunks.push(e.data);
    recorder.onstop = async () => {
        const blob = new Blob(chunks, { type: 'audio/webm' });
        const formData = new FormData();
        formData.append('audio', blob, 'gravar.webm');

        const response = await fetch('/api/transcriure', {
            method: 'POST',
            body: formData
        });
        const data = await response.json();
        console.log('Transcripció:', data.text);
    };

    recorder.start();
    // Aturar després de 5 segons (o quan l'usuari cliqui un botó)
    setTimeout(() => recorder.stop(), 5000);
}
```

El botó pot ser:

```
<button id="record">■ Parleu (5s)</button>
```

40.6 Endpoint FastAPI per transcriure

Al backend, afegim:

```
from fastapi import UploadFile, File
from faster_whisper import WhisperModel
import tempfile
import os
```



```
# Inicialitzar el model un sol cop
whisper_model = WhisperModel("large-v3", device="cpu", compute_type="int8")

@app.post("/api/transcriure")
async def transcriure(audio: UploadFile = File(...)):
    """Rep un àudio i retorna la transcripció."""
    # Guardar l'àudio temporalment
    with tempfile.NamedTemporaryFile(delete=False, suffix=".webm") as f:
        content = await audio.read()
        f.write(content)
        tmp_path = f.name

    try:
        # Transcriure
        segments, info = whisper_model.transcribe(tmp_path, language="ca")
        text = " ".join(s.text for s in segments)
        return {
            "text": text,
            "idioma": info.language,
            "probabilitat": info.language_probability
        }
    finally:
        os.unlink(tmp_path)
```

Ara el frontend pot enviar l'àudio gravat i rebre la transcripció.

40.7 Integració amb el RAG

Combinem transcripció + RAG:

```
@app.post("/api/veu-preguntar")
async def veu_preguntar(audio: UploadFile = File(...)):
    """Rep àudio, transcriu, fa consulta RAG, retorna resposta en text."""
    # 1. Transcriure
    text = await _transcriure(audio)

    # 2. Fer la consulta al RAG
    resposta = rag.ask(text['text'])

    return {
        "pregunta_transcrita": text['text'],
        "resposta": resposta['resposta'],
        "fonts": resposta['fonts']
    }
```

I podem afegir síntesi de veu a la resposta:

```
@app.post("/api/veu-preguntar-audio")
async def veu_preguntar_audio(audio: UploadFile = File(...)):
    """Mateix que veu-preguntar, però retorna la resposta en àudio."""
    text = await _transcriure(audio)
    resposta = rag.ask(text['text'])

    # Sintetitzar la resposta a veu
    audio_path = await _sintetitzar(resposta['resposta'])

    return FileResponse(
        audio_path,
        media_type="audio/mpeg",
        headers={
            "X-Pregunta": text['text'],
            "X-Fonts": ", ".join(resposta['fonts'])
        }
    )
```

)

40.8 Síntesi de veu amb Piper

Piper és un sistema de síntesi de veu local, lleuger i amb bona qualitat. Suporta català.

Instal·lació

```
pip install piper-tts
```

Descarrega un model de veu en català:

```
# Descarregar model de veu (exemple)
wget https://huggingface.co/rhasspy/piper-voices/resolve/main/ca/ca_ES/voice.json
wget https://huggingface.co/rhasspy/piper-voices/resolve/main/ca/ca_ES/voice.onnx
```

Ús

```
from piper import PiperVoice
import wave

voice = PiperVoice.load("ca_ES/voice.onnx", "ca_ES/voice.json")
text = "Hola, soc l'assistent Hort Osona. Com puc ajudar-te?"

with wave.open("resposta.wav", "wb") as f:
    voice.synthesize(text, f)
```

40.9 Síntesi alternativa: XTTS

XTTS (Coqui) permet clonar veus amb només 6 segons d'àudio. Recomanat si vols que l'assistent parli amb la teva veu o la d'un familiar.

```
pip install TTS

from TTS.api import TTS

tts = TTS("tts_models/multilingual/multi-dataset/xtts_v2", gpu=False)
tts.tts_to_file(
    text="Hola, benvingut al teu hort",
    file_path="resposta.wav",
    speaker_wav="la_meva_veu.wav", # 6 segons de la teva veu
    language="ca"
)
```

40.10 Alternativa simple: Web Speech API

Si no vols instal·lar res al backend, pots fer servir la **Web Speech API** del navegador:

```
// Reconèixer veu (STT)
const recognition = new webkitSpeechRecognition();
recognition.lang = 'ca-ES';
recognition.continuous = false;

recognition.onresult = (event) => {
    const text = event.results[0][0].transcript;
    console.log('Transcripció:', text);
};

recognition.start();

// Sintetitzar veu (TTS)
```

```
const utterance = new SpeechSynthesisUtterance("Hola món");
utterance.lang = 'ca-ES';
utterance.rate = 1.0;
speechSynthesis.speak(utterance);
```

Avantatges: zero instal·lació, funciona al navegador. Inconvenients: depèn del navegador, no sempre català de qualitat, no és privat (Google processa).

Recomanació: usa Web Speech API per començar, i migra a Whisper + Piper quan vulguis més qualitat o privadesa total.

40.11 Optimitzar el reconeixement de veu

Whisper és bo, però podem millorar-lo:

1. **Forçar idioma:** `language="ca"` evita que provi altres idiomes.
2. **Detectar silenci:** parar la gravació quan hi ha 2 segons de silenci.
3. **Filtrar soroll:** aplicar un filtre de reducció de soroll abans de transcriure.
4. **Usar el model adequat:** `small` o `medium` per a velocitat, `large-v3` per a qualitat.
5. **Ajustar la mida del vocabulari:** passa una llista de termes esperats (noms de plantes) per millorar el reconeixement.

Exemple de llista de termes:

```
from faster_whisper import WhisperModel

model = WhisperModel("large-v3", device="cpu", compute_type="int8")

segments, info = model.transcribe(
    "audio.webm",
    language="ca",
    initial_prompt="carbassa, tomaquet, enciam, mongeta, ceba, all, porro, "
                  "bleda, espinac, rave, meló, pebrot, col, escarola, "
                  "alfàbrega, menta, romaní, farigola, orenga, sajolida, "
                  "api, carabassa, patata, pastanaga, compota, purins, "
                  "adob, compost, fem, humus, reg, ascla, aixada, trasplantar"
)
```

Això ajuda Whisper a reconèixer correctament els termes específics d'horticultura.

40.12 Com crear una interfície completa de veu

Combinem-ho tot en una interfície usable:

```
<div id="voice-control">
  <button id="record-btn" class="big-btn">■</button>
  <div id="status-veu">Prem per parlar</div>
  <div id="transcripcio"></div>
  <div id="resposta"></div>
</div>

const recordBtn = document.getElementById('record-btn');
const statusVeu = document.getElementById('status-veu');
let gravant = false;
let mediaRecorder = null;
let chunks = [];

recordBtn.addEventListener('click', async () => {
```

```

    if (!gravant) {
      // Començar a gravar
      const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
      mediaRecorder = new MediaRecorder(stream);
      chunks = [];
      mediaRecorder.ondataavailable = e => chunks.push(e.data);
      mediaRecorder.onstop = enviarAudio;
      mediaRecorder.start();
      gravant = true;
      recordBtn.textContent = '■';
      statusVeu.textContent = 'Escoltant...';
    } else {
      // Aturar
      mediaRecorder.stop();
      gravant = false;
      recordBtn.textContent = '■';
      statusVeu.textContent = 'Processant...';
    }
  });

  async function enviarAudio() {
    const blob = new Blob(chunks, { type: 'audio/webm' });
    const formData = new FormData();
    formData.append('audio', blob, 'gravar.webm');

    try {
      const response = await fetch('/api/veu-preguntar', {
        method: 'POST',
        body: formData
      });
      const data = await response.json();
      document.getElementById('transcripcio').textContent =
        'Tú: ' + data.pregunta_transcrita;
      document.getElementById('resposta').textContent =
        'Assistent: ' + data.resposta;
      statusVeu.textContent = 'Fet';
    } catch (e) {
      statusVeu.textContent = 'Error: ' + e.message;
    }
  }
}

```

40.13 Privadesa de la veu

La veu és una dada **especialment sensible**: conté biometria, emocions, salut. Per això:

1. **No enviar al núvol**. Whisper local + Piper local.
2. **Xifrar l'àudio** si l'emmagatzemes.
3. **Esborrar després d'ús**. No cal guardar totes les gravacions.
4. **Permetre l'opt-out**. L'usuari ha de poder desactivar la veu.

Això és important no només per privadesa, sinó per **complir normatives** (GDPR, LOPDGDD a Espanya).

40.14 Veu offline al mòbil

Si vols que el client web funcioni **offline** al mòbil (a l'hort sense cobertura):

1. **PWA** (Progressive Web App, aplicació web que es comporta com a app nativa): converteix el client en una "app" que es pot instal·lar.
2. **Service Worker** (script que permet que la web funcioni offline): permet cachejar recursos.
3. **Whisper al mòbil**: hi ha versions (Whisper.cpp, Whisper Android).

La PWA és la solució més pràctica. Pots instal·lar el client web al mòbil amb "Add to Home Screen" i funciona com una app.

40.15 Resum

Hem après a afegir veu a l'assistent Hort Osona: transcripció amb Whisper, síntesi amb Piper o Web Speech API, integració amb el RAG, i una interfície usable. Hem vist com optimitzar el reconeixement, gestionar la privadesa, i preparar el client per a ús offline. Al proper capítol veurem les bones pràctiques de privadesa i seguretat per a sistemes d'IA local.

40.16 Exercicis pràctics

1. Instal·la faster-whisper al Mac.
2. Descarrega el model `large-v3` (o `small` si tens poca RAM).
3. Grava un àudio de prova amb el mòbil i transcriu-lo.
4. Afegeix l'endpoint `/api/transcribe` al backend del Cap 38.
5. Crea una interfície amb botó de micròfon.
6. Afegeix síntesi de veu amb Piper o Web Speech API.
7. Combina transcripció + RAG + síntesi en un sol endpoint.
8. Fes proves amb termes específics d'horticultura.

Paraules clau: **veu, STT, TTS, speech-to-text, text-to-speech, Whisper, faster-whisper, Piper, XTTS, Coqui, àudio, gravació, micròfon, MediaRecorder, Web Audio API, Web Speech API, SpeechRecognition, SpeechSynthesis, frontend, mic permission, getUserMedia, FormData, multipart, blob, webm, mp3, wav, format, codificació, privacy, GDPR, LOPDGDD, biometria, opt-out, offline, PWA, Service Worker, install, Add to Home Screen, manifest, model de veu, clonar veu, veu sintètica, idioma, català, ca-ES, optimització, transcripció, vocabulari, initial_prompt, hotwords, language detection, vad, voice activity detection, silenci, gravació, 5 segons, parlar, escoltar, streaming audio, WebSocket, àudio, format, opus, codec, compressió, qualitat, sample rate, 16kHz, 44.1kHz, micròfon, noise reduction, filtre, pre-processament, post-processament, diacritics, accent, dialecte, pronunciació.**

Capitol 41 — Privadesa i bones pràctiques

"La privadesa no és una opció, és una responsabilitat. Quan l'IA és al núvol, no saps on van les teves dades. Quan és a casa, saps exactament què passa."

41.1 Per què la privadesa importa, fins i tot en local

Pot semblar que si tens un model local, la privadesa està garantida. Però hi ha paranys:

1. **Dades d'entrada.** Encara que el model no surti del teu Mac, el que hi escrius pot quedar en logs, historial, obert en finestres, capturat per altres aplicacions.
1. **Metadades.** El fet que facis servir IA en determinats moments és informació. Si vols amagar que estàs consultant temes mèdics, o legals, o polítics, cal mesures.
1. **Dades compartides.** Si comparteixes el client amb altres persones (família, veïns), cada usuari pot veure les converses dels altres.
1. **Sortida accidental.** Un model pot filtrar informació privada en les respostes. Per exemple, si els documents contenen dades personals, el RAG els pot reproduir.
1. **Emmagatzematge.** Les converses, si les guardes, contenen dades sensibles. Cal xifrar-les o esborrar-les.

41.2 El que NO has d'enviar a una IA al núvol

Regla d'or: si una dada és prou sensible per no explicar-la a un desconegut al carrer, no l'enviïs a cap servei d'IA al núvol.

Categories específiques:

1. **Dades mèdiques:** informes, proves, historial, medicació, salut mental.
2. **Dades financeres:** números de compte, salaris, inversions, impostos.
3. **Identificadors personals:** DNI, NIE, passaport, targeta sanitària, número de la Seguretat Social.
4. **Contrasenyes i tokens:** mai, ni tan sols parcials.
5. **Correspondència privada:** cartes, missatges personals, fotos íntimes.
6. **Dades d'algú sense consentiment:** fotos d'altres, informació de tercers.
7. **Secrets comercials o professionals:** receptes, plànols, codi propietari, llistes de clients.
8. **Localització en temps real:** adreça de casa, ubicació de vacances, etc.
9. **Menors:** informació sobre nens, fotos d'escola, dades escolars.
10. **Activitat il·legal:** res que pugui ser delictu, encara que sigui per "curiositat".

Això és vàlid **fins i tot amb empreses que diuen ser privades** (Claude, ChatGPT, Gemini). Les seves polítiques poden canviar, i els seus sistemes poden tenir bretxes.

41.3 Avantatges específics de la IA local

Quan el model és a casa teva, els avantatges són clars:

1. **Cap transmissió externa.** Les dades no viatgen per Internet.
2. **Sense terme de servei.** No hi ha cap empresa que decideixi què pots o no pots fer.
3. **Sense canvis de política.** Avui ChatGPT és "privat", demà canvia la política.
4. **Inspeccionable.** Pots mirar exactament què fa el model.
5. **Personalitzable.** Pots afinar-lo amb les teves dades sense compartir-les.
6. **Funciona offline.** Sense Internet, funciona igual.

Això és especialment important per a:

- **Professionals sanitaris** que volen assistència amb casos.
- **Advocats** que treballen amb informació privilegiada.
- **Empresaris** amb dades financeres o clients.
- **Famílies** amb informació sobre menors.
- **Activistes** o periodistes en entorns hostils.

41.4 Bones pràctiques amb IA local

Encara que la IA sigui local, cal seguir bones pràctiques:

1. Configura bé el tallafocs

Assegura't que Ollama només escolta a les xarxes de confiança:

```
# Talla tot el tràfic a Ollama excepte Tailscale
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --add /Applications/Ollama.app
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --block /Applications/Ollama.app
# Permet només a la xarxa Tailscale
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblock /Applications/Ollama.app
```

2. Limita les xarxes

Si vols ser estricte, fes que Ollama escolti només a **100.64.0.0/10** (la xarxa Tailscale):

```
OLLAMA_HOST=100.115.134.76:11434 ollama serve
```

3. Autentica l'accés

Si exposa l'API a la xarxa, afegeix autenticació amb Caddy o Nginx:

```
ollama.bernat.local {
    basicauth {
        bernat $2a$14$...
    }
    reverse_proxy localhost:11434
}
```

4. Xifra l'emmagatzematge

Si el teu Mac té FileVault activat (macOS ho activa per defecte), l'emmagatzematge ja està xifrat. Si no:

```
# Activar FileVault
fdesetup enable
```

Això xifra tot el disc dur amb la teva contrasenya d'inici de sessió.

5. Neteja les dades temporals

Whisper crea fitxers temporals amb l'àudio. Assegura't que s'esborrin:

```
try:
    # Processar àudio
finally:
    if os.path.exists(tmp_path):
        os.unlink(tmp_path)
```

6. Configura la retenció de logs

Per defecte, Ollama no guarda logs de les peticions. Però el servidor web sí:

```
# A FastAPI, configura el logger per no guardar res sensible
import logging
logging.getLogger("uvicorn.access").setLevel(logging.WARNING)
```

7. Limita el context

Si el model té accés a molts documents, pot filtrar informació sensible. Configura bé la RAG per limitar l'accés.

8. Audita periòdicament

Revisa cada mes:

- Quins documents estan a la base vectorial.
- Quines converses s'han guardat.
- Qui té accés al sistema.
- Si hi ha actualitzacions de seguretat pendents.

41.5 Com gestionar converses multi-usuari

Si comparteixes el sistema amb família, amics, o veïns, cal:

1. **Autenticació per usuari.** Cada persona té el seu compte.
2. **Aïllament de dades.** Les converses d'un no es veuen per l'altre.
3. **Auditoria.** Qui ha fet què.
4. **Permisos.** Qui pot accedir a quins documents.

Implementació amb FastAPI:

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBasic, HTTPBasicCredentials
```



```

security = HTTPBasic()

USERS = {
    "bernat": "contrasenya_bernat",
    "anna": "contrasenya_anna",
    "joan": "contrasenya_joan"
}

def autenticar(credentials: HTTPBasicCredentials = Depends(security)):
    username = credentials.username
    if username not in USERS or USERS[username] != credentials.password:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Credencials incorrectes"
        )
    return username

# Cada usuari té la seva col·lecció
@app.get("/api/preguntar")
def preguntar(q: str, usuari: str = Depends(autenticar)):
    col_name = f"hort_osona_{usuari}"
    # ...

```

Ara cada usuari té el seu aïllament.

41.6 Com evitar que el RAG filtri informació sensible

Si els documents d'Hort Osona contenen dades personals, el RAG les pot reproduir. Per evitar-ho:

1. **Neteja els documents abans d'indexar.** Esborra noms, adreces, telèfons, DNI.
2. **Filtra els fragments retornats.** Si un fragment conté dades sensibles, exclou-lo.
3. **Limita l'accés per usuari.** Cada usuari té la seva vista.
4. **Marca la informació sensible.** Un sistema de tags per evitar la indexació.

Exemple de filtratge:

```

def es_sensible(text: str) -> bool:
    """Detecta si un text conté dades sensibles."""
    indicadors = [
        "DNI", "NIE", "targeta", "contrasenya",
        "@gmail.com", "@hotmail.com", "telèfon",
        "adreça", "carrer", "número"
    ]
    text_lower = text.lower()
    return any(ind.lower() in text_lower for ind in indicadors)

def cerca_filtrada(consulta: str, k: int = 4) -> list:
    resultats = collection.query(query_texts=[consulta], n_results=k*2)
    fragments_net = []
    for i, frag in enumerate(resultats['documents'][0]):
        if not es_sensible(frag):
            fragments_net.append({
                "text": frag,
                "font": resultats['metadatas'][0][i]['source'],
                "distancia": resultats['distances'][0][i] if 'distances' in resultats else 0
            })
        if len(fragments_net) >= k:
            break
    return fragments_net

```

41.7 Com gestionar el "dret a l'oblit"

Si un usuari vol esborrar totes les seves dades, cal:

1. **Esborrar la seva col·lecció vectorial.**
2. **Esborrar el seu historial de converses.**
3. **Esborrar els seus logs.**
4. **Confirmar per escrit** que s'ha fet.

Script d'esborrat:

```
def oblidar_usuari(usuari: str):
    """Esborra totes les dades d'un usuari."""
    # Esborrar col·lecció
    col_name = f"hort_osona_{usuari}"
    try:
        client.delete_collection(col_name)
    except:
        pass

    # Esborrar historial
    historial_path = Path(f"./historial/{usuari}.json")
    if historial_path.exists():
        historial_path.unlink()

    # Esborrar logs
    logs_path = Path(f"./logs/{usuari}.log")
    if logs_path.exists():
        logs_path.unlink()

    print(f"Dades de {usuari} esborrades correctament")
```

41.8 Bones pràctiques amb prompts

A l'hora de fer prompts, pensa:

1. **No comparteixis informació personal identificable (PII)** si no és estrictament necessari.
2. **Usa noms falsos** per a persones en exemples.
3. **Esborra la informació sensible** de les respostes del model abans de desar-les.
4. **Configura el system prompt** perquè el model eviti inventar dades personals:

Si et demanen informació personal d'algú, respon "No tinc aquesta informació" en lloc d'inventar-la.

41.9 Com auditar el sistema

Un cop al mes, revisa:

1. **Quins documents estan indexats:** `ls vectorstore/`
2. **Quines converses s'han desat:** `ls historial/`
3. **Quins logs hi ha:** `tail logs/assistant.log`
4. **Qui ha accedit:** revisar logs d'autenticació

5. Quines actualitzacions hi ha: `ollama list, pip list --outdated`

Un script d'auditoria:

```
def auditar_sistema():
    print("=== Auditoria mensual del sistema ===\n")
    print(f"Documents indexats: {collection.count()}")
    print(f"Converses guardades: {len(list(Path('historial').glob '*.json'))}")
    print(f"Mida de la base vectorial: {sum(f.stat().st_size for f in Path('vectorstore').rglob('*')) / 1024}")
    print(f"Mida de l'historial: {sum(f.stat().st_size for f in Path('historial').rglob('*')) / 1024}")
    # Més coses...
```

41.10 Què fer si hi ha una bretxa

Si sospites que algú ha accedit al sistema sense permís:

1. **Desconnecta l'API** (`kill ollama`).
2. **Canvia les contrasenyes** (Tailscale, sistema, etc.).
3. **Revisa els logs** per veure què s'ha fet.
4. **Notifica als afectats** si hi ha dades personals.
5. **Reinstal·la Ollama** des de zero.
6. **Re-indexa** els documents.
7. **Documenta la incidència** al README.

41.11 Compliment normatiu

A Espanya i la UE, el Reglament General de Protecció de Dades (RGPD) i la Llei Orgànica de Protecció de Dades (LOPDGDD) apliquen. Amb IA local, tens molt bon posicionament, però encara cal:

1. **Consentiment informat**: si altres persones usen el sistema.
2. **Transparència**: explicar què es fa amb les dades.
3. **Minimització**: només recull el que necessitis.
4. **Limitació de la finalitat**: no usar les dades per a altres coses.
5. **Integritat i confidencialitat**: protegir-les.
6. **Responsabilitat proactiva**: poder demostrar que compleixes.

Documenta tot això en una **Política de Privacitat** si el sistema és compartit.

41.12 Bones pràctiques amb el backup

Quan fas còpies de seguretat, xifra-les:

```
# Backup xifrat amb gpg
tar -czf - ~/bernatlab/asistent/ | gpg -c > backup-asistent-$(date +%Y%m%d).tar.gz.gpg
```

Això xifra la còpia amb una contrasenya. Guarda la contrasenya en un gestor de contrasenyes (1Password, Bitwarden, KeePass).

41.13 Resum

Hem après les bones pràctiques de privadesa per a sistemes d'IA local: què no enviar al núvol, com configurar el tallafocs i l'autenticació, com gestionar múltiples usuaris, com evitar filtracions del RAG, com auditar, i com complir la normativa. Al proper capítol veurem 10 consultes reals que pots fer a l'assistent Hort Osona per començar a usar-lo de manera pràctica.

41.14 Exercicis pràctics

1. Fes una llista de quines dades personals tens a les fitxes d'Hort Osona.
2. Configura el tallafocs del Mac per permetre Ollama només des de Tailscale.
3. Activa FileVault si no el tens activat.
4. Implementa autenticació bàsica al backend.
5. Configura la neteja automàtica de fitxers temporals.
6. Crea un script d'auditoria mensual.
7. Documenta al README la política de privadesa del sistema.

Paraules clau: **privadesa, privacy, GDPR, RGPD, LOPDGDD, dades personals, PII, identificació, biometria, consentiment, transparència, minimització, finalitat, integritat, confidencialitat, responsabilitat proactiva, política de privadesa, tallafocs, firewall, FileVault, xifrat, encryption, autenticació, basicauth, OAuth, JWT, multi-usuari, aïllament, retenció de dades, dret a l'oblit, esborrat, compliança, auditoria, logs, monitoratge, alerting, backup xifrat, gpg, contrasenya, gestor de contrasenyes, KeePass, Bitwarden, 1Password, model local, Ollama, Tailscale, xarxa privada, IP, tallafocs per IP, subnet, ACL, MAC filtering, port forwarding, NAT, port, 11434, 8080, 0.0.0.0, localhost, 100.x, 192.168, mTLS, TLS, certificat, client certificate, mutual TLS, fingerprint, verificació, prompt injection, sanitization, validació, escape, sandbox, contenidor, Docker, aïllament, recursos, cgroup, namespace, seccomp, AppArmor, SELinux, hardening, CIS, NIST, security baseline, OWASP, top 10, application security, dependency, vulnerabilitat, CVE, update, patch, manteniment.**

Capítol 42 — 10 consultes reals a l'assistent Hort Osona

"Ja tens l'eina. Ara veurem què li pots preguntar per començar a treure'n profit des del primer dia."

42.1 Com usar aquest capítol

Aquest capítol és diferent dels altres. No és teoria ni pràctica, sinó **exemples reals** de consultes que pots fer a l'assistent Hort Osona un cop estigui muntat (Caps 33-41).

Cada consulta inclou:

- La **pregunta exacta** que pots escriure o dictar.
- El **context** (per què és útil).
- La **resposta esperada** (orientativa — dependrà del model que usis).
- Els **documents** que el RAG hauria de recuperar.

Al final tens un **prompt de benvinguda** que pots personalitzar.

42.2 Consulta 1: el calendari de sembra del mes

Pregunta: "Què he de sembrar aquest mes a l'hort d'Osona?"

Context: cada mes hi ha sembres òptimes. Vols saber-les sense haver de cercar per totes les fitxes.

Resposta esperada (orientativa):

Aquest mes a la comarca d'Osona (zona mitjana, ~600 m d'altitud), pots sembrar:

1. ***Enciam, escarola, ruca** (tolerant al fred).*
2. ***Pèsols, faves** (lleguminoses que volen temperatures fresques).*
3. ***Espinac, bleda** (hoja verda de cicle curt).*
4. ***Rave, pastanaga** (d'arrel, cicle ràpid).*
5. ***All, ceba tendra** (bulbs que es cullen a l'estiu).*

Consulta la fitxa de sembra per a cada cultiu al calendari d'Hort Osona. Recorda que aquesta llista és per a la zona d'Osona; altres zones poden variar 2-4 setmanes.

Fonts esperades: `calendari-sembrar.md`, `pla-mensual.md`, fitxes individuals dels cultius esmentats.

42.3 Consulta 2: diagnòstic d'una plaga

Pregunta: "Les fulles del tomàquet tenen taques grogues amb un pols blanc a sota. Què és i què puc fer?"

Context: detectes un problema a la planta i vols consell ràpid.

Resposta esperada:

*Pel que descrius, és probable que sigui ****míldiu**** (*Phytophora infestans*), un fong que afecta tomàquets i patates, especialment amb humitat alta. Confirma mirant si:*

- 1. Les taques s'escampen ràpidament.*
- 2. A sota hi ha una mena de pelussa blanca.*
- 3. Les tiges també es taquen.*

****Tractament ecològic**:**

- ****Preventiu**:** sulfat de coure (0.3-0.5%) o decocció de cua de cavall.
- ****Curatiu**:** infusió d'all o purí d'ortiga.
- ****Eliminar**** fulles afectades i cremar-les (no compostar).
- ****Ventilació**:** esporgar les plantes per millorar la circulació d'aire.
- ****Evitar**** reg per aspersió; millor reg per degoteig.

Si la infecció és greu, és possible que hakis de sacrificar la planta. Consulta la fitxa de plagues per a més detalls.

Fonts esperades: [gestio-plagues.md](#), [fitxa-tomaque.md](#), [practiques-avancades.md](#).

42.4 Consulta 3: associacions de cultius

Pregunta: "Vull fer un hort de 100 m² amb tomàquets, enciams i pastanagues. Quines associacions em recomanes?"

Context: estàs planificant un hort nou i vols optimitzar l'espai.

Resposta esperada:

Per a un hort de 100 m² amb tomàquets, enciams i pastanagues, et recomano:

****Associacions positives**** (ajuden mútuament):

- 1. ****Tomàquet + alfàbrega****: l'alfàbrega repel·la els mosquits i millora el gust del tomàquet.*
- 2. ****Enciam + pastanaga****: l'enciam ombreja el sòl, mantenint la humitat que la pastanaga necessita.*
- 3. ****Tomàquet + ceba/all****: la ceba repel·la plagues del tomàquet.*

****Rotació recomanada**** (any següent):

- *On hi havia tomàquet, plantar lleguminoses (faves, pèsols) per fixar nitrogen.*

- On hi havia enciam, plantar brassicàcies (col, rave).

- On hi havia pastanaga, plantar tomàquet.

****Disposició suggested****:

...

[Tomàquets 60m²] [Enciams 20m²] [Pastanagues 20m²]

[Alfàbrega intercalada] [Ceba intercalada]

...

Consulta la guia d'associacions per a més combinacions.

Fonts esperades: associacions-rotacions.md, croquis-hort.md, fitxes individuals.

42.5 Consulta 4: com fer compost casolà

Pregunta: "Com puc fer compost a casa amb restes de cuina si visc en un pis?"

Context: vols compost però no tens jardí.

Resposta esperada:

Per fer compost en un pis, la millor opció és un ****composter domèstic**** tipus ****Bokashi**** (fermentació anaeròbica en galleda tancada) o un ****composter de balcó**** amb cucs.

****Opció 1: Bokashi**** (recomanada per a pisos):

1. Compra una galleda Bokashi (~30 €).
2. Afegeix restes de cuina (fruita, verdura, pa, marro de cafè).
3. Escampa una capa de micelis Bokashi cada 2 cm de restes.
4. Premsa per expulsar l'aire.
5. Tanca la tapa. Cada 2-3 dies, obre la vàlvula per escórrer el líquid (dilueix 1:100 en aigua i usa com a adob líquid).
6. En 2-4 setmanes, el compost estarà fermentat.

****Opció 2: Vermicompostador amb cucs**** (per a balcons):

1. Compra un vermicompostador (~50 €).
2. Afegeix restes vegetals (no carn ni làctics).
3. Els cucs (*Eisenia foetida*) les transformen en humus.
4. Cada 3-6 mesos, recull l'humus del compartiment inferior.

****Important****:

- No posis carn, peix, làctics, ni olis.
- Evita cítrics en grans quantitats (els cucs no els toleren).
- Mantén humit però no enxarcat.

Consulta la fitxa de compost per a més detalls.

Fonts esperades: compost.md, practiques-avancades.md.

42.6 Consulta 5: poda de fruiters

Pregunta: "Quan i com he de podar els meus presseguers de 5 anys?"

Context: tens fruiters joves i vols podar correctament.

Resposta esperada:

*Els ****presseguers**** (*Prunus persica*) es poden a ****final d'hivern o inici de primavera****, quan ja no hi ha risc de gelades fortes però abans que brotin.*

****Objectius de la poda****:

1. ****Eliminar branques mortes o malaltes****.
2. ****Obrir la capçada**** perquè entri llum i aire.
3. ****Renovar branques fructíferes**** (el presseguer fructifica en branques de l'any anterior).

****Tipus de branques a distingir****:

- ****Brindilles**** (curtes, amb borrons arrodonits): fructifiquen.
- ****Branques mixtes**** (mitjanes): fructifiquen parcialment.
- ****Brots d'aigua**** (llargs i verticals): no fructifiquen, treure.

****Passos****:

1. ****Eines****: tisores de poda netes i afilades, serra petita per branques grosses.
2. ****Cortes a 45°****, just per sobre d'un borró que miri cap a fora.
3. ****Eliminar**** brots d'aigua, branques creuades, xucladors.
4. ****Deixar**** 3-4 branques principals ben distribuïdes.
5. ****Aplica pasta segelladora**** en talls grans.

****Consell****: la primera poda en arbres joves és la més important. Defineix bé l'estructura els primers 3 anys.

Fonts esperades: [fruiters-guia-completa.md](#), [practiques-avancades.md](#).

42.7 Consulta 6: reg durant la calor

Pregunta: "Estem passant una onada de calor a 38 °C. Com he de regar l'hort?"

Context: temps extrem, vols evitar pèrdues.

Resposta esperada:

Amb temperatures superiors a 35 °C, cal ajustar el reg:

****Horari**:**

- Rega ****primerenca del matí**** (6-9 h) o al ****vespre**** (20-22 h).
- ****Mai**** al migdia (l'aigua s'evapora i pot cremar les fulles).

****Freqüència**:**

- Hortalisses de fruit (tomàquet, pebrot, albergínia): ****cada 1-2 dies****.
- Hortalisses d'arrel (pastanaga, rave): ****cada 2-3 dies****.
- Hortalisses de fulla (enciam, espinac): ****diari****.
- Fruïters: ****setmanal**** amb reg profund.

****Quantitat**:**

- 10-15 litres per m² en regs profunds.
- 5-8 litres per m² en regs de manteniment.

****Tècniques d'estalvi**:**

- ****Encoixinat**** (mulching, cobrir el sòl amb palla, fullaraca, o plàstic) redueix l'evaporació un 50-70%.
- ****Reg per degoteig****: més eficient que l'aspersió.
- ****Reg subterrani****: tubs perforats sota la superfície.

****Senyals d'estrès hídric**:**

- Fulles pansides al matí (no pas al migdia, és normal).
- Color verd pàl·lid o groguenc.
- Fruits petits o amb cremades solars.

Consulta la fitxa de reg per a cada cultiu.

Fonts esperades: [reg-fertilitzacio.md](#), fitxes individuals.

42.8 Consulta 7: control del pugó

Pregunta: "Tinc molt de pugó a les faves. Com ho combatre sense productes químics?"

Context: plaga comuna, vols solució ecològica.

Resposta esperada:

*El ****pugó**** a les faves és molt comú a la primavera. Combatre'l de forma ecològica:*

*****Tractaments directes****:*

1. *****Sabó potàssic****: 10-15 ml per litre d'aigua. Polvoritza al matí o al vespre. Repetir cada 5-7 dies.*
2. *****Infusió d'all****: 50 g d'all picat per litre d'aigua, deixa reposar 24 h, filtra i polvoritza.*
3. *****Purí d'ortiga****: fermenta 1 kg d'ortigues en 10 L d'aigua durant 1-2 setmanes. Dilueix 1:10.*
4. *****Oli de neem****: 5 ml per litre. Molt eficaç, però car.*

*****Control biològic****:*

- *****Marietes**** (Coccinellidae): mengen 50-100 pugs al dia. ****No les matis!*****
- *****Crysoperla**** (crisopa): les larves mengen molts pugs.*
- *****Sírfids****: les larves són depredadores.*

*Per afavorir aquests insectes, planta ****flors**** al voltant de l'hort: calèndula, caps d'ase, alfalfa, facèlia.*

*****Prevenició****:*

- *****Evitar**** l'excés de nitrogen (adobs massa rics atrauen pugs).*
- *****Rotació**** de cultius.*
- *****Eliminar**** herbes hoste (com la corretjola).*

Si la infestació és molt forta, poda les parts més afectades i crema-les.

Fonts esperades: [gestio-plagues.md](#), [practiques-avancades.md](#).

42.9 Consulta 8: guardar llavors

Pregunta: "Com guardo les llavors dels tomàquets per sembrar l'any vinent?"

Context: vols autosuficiència.

Resposta esperada:

Per guardar llavors de tomàquet per sembrar l'any vinent:

****1. Tria les millors plantes**:**

- Les més sanes, productives, i representatives de la varietat.
- No agafis llavors de plantes híbrides (F1): la descendència no serà fidel.

****2. Extracció de llavors** (per tomàquets):**

1. Talla el tomàquet madur per la meitat.
2. Buida les llavors amb el gel en un colador.
3. Frega-les suaument per separar la polpa.
4. Deixa-les fermentar 2-4 dies en un got amb aigua (per eliminar la capa gelatinosa).
5. Renteu-les bé.
6. Escampar-les en un paper o plàstic per assecar-les (7-10 dies).

****3. Assecatge i emmagatzematge**:**

- Asseca-les en un lloc ventilat, sense sol directe.
- Guardar-les en un ****pot de vidre**** o ****sobre de paper**** en un lloc ****fresc, sec i fosc****.
- Afegeix un ****dessecant**** (gel de sílice) per evitar humitat.
- Durada: 4-6 anys (depenent de la varietat).

****4. Etiqueta sempre**:**

- Nom de la varietat.
- Data de collita.
- Procedència (any, lloc).

****5. Prova de germinació** (opcional):**

- 10 llavors en un paper humit.
- 7-14 dies.
- Si germinen >7, la llavor és bona.

Consulta la guia de llavors per a més detalls.

Fonts esperades: [guardar-llavors.md](#).

42.10 Consulta 9: trasplantar tomàquets

Pregunta: "Quan i com trasplanto les plantes de tomàquet del planter al camp?"

Context: arriba el moment del trasplantament.

Resposta esperada:

*El trasplantament del tomàquet es fa a la ****primavera****, quan ja no hi ha risc de gelades i el sol està a ****15-18 °C**** com a mínim. A Osona, sol ser entre ****finals d'abril i mitjans de maig****.*

****Passos****:

****1. Enduriment**** (7-10 dies abans):

- Surt les plantes al balcó o fora, gradualment (1 h, 2 h, mig dia, dia sencer).
- Evita sol directe les primeres hores.
- Redueix el reg per estrès controlat.

****2. Preparació del sòl****:

- Llaura o cavar la terra a 30 cm de fondària.
- Incorpora compost madur (5-10 L/m²).
- Nivella i marca les línies de plantació.

****3. Distàncies****:

- Entre plantes: ****50-60 cm****.
- Entre línies: ****80-100 cm****.

****4. Trasplantament**** (al vespre o dia ennuvolat):

1. Rega el planter 1-2 h abans.
2. Fes forats de la mida del test.
3. Treu la planta amb el pa de terra intacte.
4. Col·loca-la al forat, fins a les primeres fulles.
5. Omple amb terra, premsant suaument.
6. Rega abundantment.

****5. Cura post-trasplantament****:

- Rega diàriament la primera setmana.

- *Protegeix del sol directe els primers 2-3 dies (tela d'ombra o campana).*
- *Col·loca tutors si la varietat ho necessita.*

***Senyals d'estrès**:* fulles pansides que es recuperen al vespre és normal. Si no es recuperen, rega més.

Fonts esperades: [calendari-sembrar.md](#), [fitxa-tomaque.md](#), [practiques-avancades.md](#).

42.11 Consulta 10: elaboració d'una conserva

Pregunta: "Com faig conserva de tomàquet per a tot l'any?"

Context: tens excedent de tomàquets i vols conservar-los.

Resposta esperada:

Per fer conserva de tomàquet casolana:

***Ingredients**:*

- *5 kg de tomàquets madurs.*
- *1 ceba gran (opcional).*
- *1 cullerada de sal.*
- *1 cullerada de sucre (opcional, per reduir acidesa).*
- *1 cullerada d'oli d'oliva.*
- *Alfàbrega fresca (opcional).*

***Passos**:*

1. ***Rentar** i **tallar** els tomàquets a quarts.*
2. ***Bullir** 30-45 minuts a foc lent, tapats, fins que es desfacin.*
3. ***Triturar** amb batedora o passar pel passapures.*
4. ***Colar** per eliminar llavors i pell (opcional, segons el gust).*
5. ***Coure** 20-30 minuts més, afegint sal, sucre, ceba picada, alfàbrega.*
6. ***Provar** i rectificar de sal.*
7. ***Envasar** en pots de vidre esterilitzats.*
8. ***Tapar** i **bullir al bany maria** 30 minuts per segellar.*
9. ***Deixar refredar** i guardar en lloc fresc i fosc.*

***Durada**:* 1-2 anys si es conserva correctament.

****Consells**:**

- Esterilitza els pots al forn a 120 °C 15 min.
- Assegura't que la tapa fa "click" quan es refreda (bona soldadura).
- Si una tapa no fa "click", guardar a la nevera i consumir aviat.

****ATENCIÓ**:** un pot mal soldat pot desenvolupar ****botulisme**** (perill mortal). Si el tomàquet fa mala olor, color estrany, o gas en obrir-lo, llença'l.

Fonts esperades: [conserves.md](#), [fitxa-tomaque.md](#).

42.12 Prompt de benvinguda personalitzat

Quan configuris el teu assistent, pots personalitzar el system prompt:

Ets l'assistent Hort Osona. Coneixes les 76 fitxes de cultius del projecte BernatLab i les guies d'horticultura ecològica de la comarca d'Osona.

CARACTERÍSTIQUES:

- Respon SEMPRE en català.
- Sigues pràctic i directe, sense floritures.
- Dona consells adaptats a la comarca d'Osona (zona mitjana, ~600 m, clima continental).
- Prioritza mètodes ecològics i sostenibles.
- Si no saps la resposta, digues "No tinc prou informació a les fitxes d'Hort Osona. Consulta la fitxa corresponent o pregunta al fòrum".
- Cita sempre les fonts al final de la resposta.

ESTIL:

- Frases curtes.
- Llistes numerades quan hi ha diversos punts.
- Negreta per termes clau.
- Màxim 10-15 línies per resposta (o menys).

Aquest prompt personalitzat es posa a la configuració d'Ollama (Cap 34) o a la variable d'entorn del backend.

42.13 Consells finals

1. **Pregunta amb context.** "Com plantar carbasses?" és menys útil que "Com plantar carbasses en un hort de 50 m² a Osona amb sòl argilós?".

1. **Confirma les respostes.** El model pot inventar coses. Si una recomanació et sembla estranya, consulta la fitxa original.

1. **Fes-lo servir sovint.** Com més el facis servir, més aprendràs a formular bones preguntes.

1. **Ensenya'l a la família.** Si comparteixes l'hort amb algú, ensenya'ls a usar l'assistent. Així tothom hi contribueix.

1. **Documenta les respostes bones.** Si el model dona una resposta excel·lent, guarda-la. Pot servir d'inspiració per a futures consultes.

42.14 Resum

Aquest capítol ha presentat 10 consultes reals que pots fer a l'assistent Hort Osona: el calendari de sembra, diagnòstic de plagues, associacions, compost, poda, reg, plagues, guardar llavors, trasplantar, i conserves. Cada consulta ha mostrat la pregunta, el context, la resposta esperada, i les fonts. També hem vist un prompt de benvinguda personalitzat per configurar el teu assistent.

Aquest és l'últim capítol del Mòdul 4. Ara tens tot el que necessites per tenir un assistent d'IA local potent, privadesa garantida, i totalment integrat amb el teu hort. Comença amb una pregunta senzilla i experimenta!

42.15 Exercicis pràctics

1. Tria 3 de les 10 consultes d'aquest capítol i fes-les al teu assistent (quan estigui muntat).
2. Avalua la qualitat de les respostes i compara-les amb les respostes esperades.
3. Personalitza el system prompt amb les teves pròpies instruccions.
4. Documenta al README les 5 millors respostes que has obtingut.
5. Comparteix les consultes útils amb família o veïns.

Paraules clau: **consultes, exemples, casos d'ús, calendari de sembra, plagues, associacions, compost, poda, reg, conserves, llavors, trasplantament, hort Osona, comarca d'Osona, primavera, estiu, tardor, hivern, calendari lunar, rotació, conreu associat, planter, hivernacle, microclima, adob, fertilitzant, nitrogen, fòsfor, potassi, reg per degoteig, aspersió, encoixinat, mulching, conserves, fermentació, lactofermentació, oli d'oliva, alfàbrega, sajolida, farigola, orenga, plantes aromàtiques, plantes medicinals, infusions, decoccions, maceracions, ungüents, cremes, tintures, remeieres, saviesa popular, transmissió, aprenentatge, oralitat, etnografia, memòria oral.**